HexaStock Consulting Guide

HexaStock Consulting Guide

Spring Modulith • Domain Events

Watchlists • Hexagonal Architecture

Large-format reading edition for reMarkable 2

Branch:
`feature/modulith-watchlists-extraction`

Generated: 2026-04-25

Contents

Monday Session – Documentation Map **10**

File index 10

Diagram index 16

Recommended reading order (for the instructor) . 17

Cross-references to existing material 18

00 – Architecture Overview **20**

1. What HexaStock is 20
2. Business capabilities (~ candidate bounded contexts) 21
3. The four architectural styles in play, and how they coexist 22
   - 3.1 DDD – strategic and tactical 24
   - 3.2 Hexagonal Architecture – ports and adapters 24
   - 3.3 Spring Modulith – runtime enforcement of bounded contexts 25
   - 3.4 Domain Events – the only intentional decoupling mechanism today 26
4. Why this combination makes sense *here* 27
5. What this overview is *not* 28

01 – Filesystem, Maven and Modulith Structure **29**

1. The three orthogonal axes 29

2. Maven multi-module layout 30
Why split adapters into separate Maven modules at all? 32
3. Filesystem layout 33
4. The relationship between Maven modules, Java packages, and Modulith modules . . . 35
5. The Spring Modulith view 37
6. How the layout supports – and constrains – DDD and Hexagonal 39
Supports 39
Constrains 40

02 – Watchlists / Market Sentinel: Domain Event Deep Dive 41

1. Business intent 41
2. End-to-end sequence 42
3. The event itself – anatomy of a good domain event 45
4. Where the event lives – and why that matters 47
5. Transactional semantics – the real reason `@ApplicationModuleListener` exists 50
6. The notification side – why it can be ignored from the publisher's point of view 52
7. Why this is a perfect teaching example 53
8. Caveats and honest limitations 55

03 – Layout Alternatives: a Rigorous Comparison 57
A. Layered, single Maven module 57
B. Package-by-feature, single Maven module . . . 59
C. Per-bounded-context mini-hexagons (Maven module per BC) 60
D. Modulith-first (top-level package per module, single Maven module) 62
E. HexaStock today – Hexagonal × Bounded Context, by Maven *and* by package 63
How to explain this coherently to senior engineers 65

04 – From Current Solution to Realistic Production Evolution 67
1. What the current implementation actually guarantees 68
2. Evolution path 1 – add durability inside the same monolith 69
3. Evolution path 2 – externalise selected events to a broker 71
4. Evolution path 3 – physically extract a module 73
5. What the current architecture deliberately *does not* claim 74
6. Where this leaves the consultancy thesis . . . 75

Exercise – Refactor *Sell Stocks* with Domain Events **76**
At a glance 76
1. Why this exercise exists 78
2. The instructor demo: the existing Market Sentinel flow 79
3. The current *Sell Stocks* implementation . . . 81
4. The architectural problem 83
4.1 Open-closed violation under future reactions 84
4.2 Transactional coupling between the decision and the reaction 84
4.3 Bounded-context leakage as the platform grows 85
5. The exercise 85
5.1 Required outcome 86
5.2 Open design questions for the participants 87
5.3 Suggested package layout for the proposed solution 89
5.4 Suggested event shape 90
5.5 Suggested publication site 92
5.6 Suggested consumer 93
5.7 Suggested testing 95
6. Hexagonal-architecture boundaries to preserve 97
7. Stretch goal: the canonical *"one operation, many events"* pattern 98

8. Out of scope (deliberate) 100
9. Trade-offs the review must address 101
10. Instructor notes 103
10.1 Preparation 103
10.2 Suggested timing (half-day session) . . 103
10.3 What to look for during the review . . 105
10.4 Common pitfalls 106
11. Where this fits in the wider documentation set 107
12. Acceptance checklist for the participants' deliverable 108

05 – Instructor Guide for the Monday Session 111
1. Session objective 111
2. Suggested agenda (half-day, ~3.5 h with breaks) 112
3. Recommended flow 114
3.1 Open with the *thesis*, not with the project 114
3.2 Tour the filesystem live, with the IDE projector 114
3.3 Run the verification tests live 115
3.4 Run the integration test as the demo . . 115
3.5 Do the layout-alternatives discussion as a discussion, not a lecture 116
3.6 Set up the exercise *before* the break . . 116
4. Key talking points (memorise three of these, improvise the rest) 117
5. Demo sequence (cut-and-keep card) 118

6. Discussion prompts 119
7. Common misconceptions to address proactively 119
8. Architecture trade-offs to highlight (when asked) 121
9. Suggested exercise flow (35 minutes) 122
10. Wrap-up points (last five minutes) 123

06 – Slide Deck Specification (AI-feedable) 124

How to feed this to a slide generator 125
Slide 1 125
Slide 2 126
Slide 3 127
Slide 4 128
Slide 5 128
Slide 6 129
Slide 7 130
Slide 8 130
Slide 9 131
Slide 10 132
Slide 11 133
Slide 12 133
Slide 13 134
Slide 14 135
Slide 15 136
Slide 16 136
Slide 17 137
Slide 18 138

Slide 19 139
Slide 20 139
Slide 21 140
Slide 22 141
Slide 23 142
Slide 24 142
Slide 25 143
Slide 26 144
Slide 27 144
Slide 28 145
Notes for the slide generator 146

Monday Session – Documentation Map

Scope. This folder is a *focused* documentation set produced for the upcoming consultancy / training session that uses HexaStock as a teaching asset for **DDD + Hexagonal Architecture + Spring Modulith + Domain Events**. It complements, but does not replace, the broader `doc/consultancy/` briefing pack and the `doc/architecture/` reference material.

Branch. All material lives on `feature/modulith-watchlists-extraction`. Nothing here ships to `main`.

File index

#	File	Type	Purpose
0	README.md	Index	This map and reading order.
1	00-ARCHITECTURE-OVERVIEW.md	Main doc	What HexaStock is, business capabilities, the four architectural styles in play, why the combination makes sense here.

#	File	Type	Purpose
2	01-FILESYSTEM-AND-MAVEN-STRUCTURE.md	Main doc	Filesystem layout, Maven multi-module, package structure, how Maven modules / packages / Modulith modules relate.
3	02-WATCHLISTS-EVENT-FLOW-DEEP-DIVE.md	Main doc	End-to-end walk-through of `WatchlistAlertTriggeredEvent`, why it is the canonical teaching example.

#	File	Type	Purpose
4	03-LAYOUT-ALTERNATIVES.md	Main doc	Rigorous comparison of the current layout against four common alternatives.
5	04-PRODUCTION-EVOLUTION.md	Main doc	Realistic evolution paths beyond the current in-process domain events: outbox, externalisation, eventual extraction.

#	File	Type	Purpose
6	05-INSTRUCTOR-GUIDE.md	Teaching	Practical guide for running the Monday session: agenda, talking points, demo sequence, discussion prompts, exercise flow.
7	06-SLIDE-DECK-SPEC.md	Teaching	Structured, AI-feedable slide-by-slide specification.

#	File	Type	Purpose
8	diagrams/	Sources	12 PlantUML sources for this folder, each rendered to PNG and SVG under `diagrams/Rendered/`.

The Sell Stocks event-driven exercise also gets three new diagrams – they live with the existing tutorial under `doc/tutorial/sellStocks/diagrams/`, following the convention already established in that folder:

File	Purpose
diagrams/sell-events-current.puml	Sequence – current synchronous sell flow.
diagrams/sell-events-target.puml	Sequence – target event-driven sell flow.
diagrams/sell-events-conceptual.puml	Conceptual – one business fact, many reactions.

The exercise document itself, SELL-STOCK-DOMAIN-EVENTS-EXERCISE.md, is updated in-place to embed the three rendered diagrams.

Diagram index

All diagram sources are PlantUML. Each `.puml` has a sibling `Rendered/<name>.svg` and `Rendered/<name>.png` produced by `scripts/render-diagrams.sh` (Docker image `plantuml/plantuml:latest`).

#	Source	Rendered
1	01-architecture-overview.puml	PNG · SVG
2	02-maven-multimodule.puml	PNG · SVG
3	03-filesystem-layout.puml	PNG · SVG
4	04-modulith-modules.puml	PNG · SVG
5	05-bounded-context-map.puml	PNG · SVG
6	06-hexagonal-view.puml	PNG · SVG
7	07-watchlist-sentinel-sequence.puml	PNG · SVG
8	08-watchlist-event-flow.puml	PNG · SVG
9	09-notification-flow.puml	PNG · SVG
10	10-layout-alternatives.puml	PNG · SVG
11	11-current-vs-future-events.puml	PNG · SVG

#	Source	Rendered
12	12-watchlist-rest-sequence.puml	PNG · SVG
S1	sell-events-current.puml	PNG · SVG
S2	sell-events-target.puml	PNG · SVG
S3	sell-events-conceptual.puml	PNG · SVG

To regenerate everything from the repository root:

```
./scripts/render-diagrams.sh
```

Recommended reading order (for the instructor)

Read in this order to prepare for Monday – total approx. 90 minutes:

1. **00-ARCHITECTURE-OVERVIEW.md** – anchor the mental model.
2. **01-FILESYSTEM-AND-MAVEN-STRUCTURE.md** – be ready to explain *where things live and why.*
3. **02-WATCHLISTS-EVENT-FLOW-DEEP-DIVE.md** – the demo case.

4. **03-LAYOUT-ALTERNATIVES.md** – the part where senior engineers will push back; have the trade-offs internalised.
5. **04-PRODUCTION-EVOLUTION.md** – the "what would we do at real scale?" conversation.
6. **../../tutorial/sellStocks/SELL-STOCK-DOMAIN-EVENTS-EXERCISE.md** – the participant exercise.
7. **05-INSTRUCTOR-GUIDE.md** – agenda and timing.
8. **06-SLIDE-DECK-SPEC.md** – feed into your slide tool of choice.

Cross-references to existing material

- Higher-level briefing chapters: `doc/consultancy/` (DDD, Hexagonal, Modulith, Domain Events deep dive, Domain Events roadmap, cheatsheet).
- Bounded-context inventory and migration history: `doc/architecture/MODULITH-BOUNDED-CONTEXT-INVENTORY.md`.
- Architectural decision records: `doc/architecture/adr/`.

- Hexagonal layering reference diagrams: `doc/tutorial/sellStocks/HEXAGONAL-ARCHITECTURE-REFERENCE-DIAGRAMS.md`.
- Ubiquitous language: `doc/tutorial/sellStocks/UBIQUITOUS-LANGUAGE.md`.

00 – Architecture Overview

Audience. Senior Java engineers and architects attending the Monday session. **Reading time.** ~15 minutes. **Prerequisite.** Working knowledge of Spring Boot and basic DDD vocabulary.

---

1. What HexaStock is

HexaStock is a small but realistic financial-portfolio backend used inside the Generalitat de Catalunya / AGAUR teaching context. Functionally it lets a user:

- open a **Portfolio**, deposit and withdraw cash,
- **buy** stocks at the current market price (creating *lots* with FIFO ordering),
- **sell** stocks consuming those lots in FIFO order and computing realised profit / loss,
- inspect **Transactions** (an append-only ledger of all monetary movements),
- maintain **Watchlists** with price-threshold alerts, and receive a **notification** when an alert triggers.

Stock prices come from external market-data providers (Finnhub, Alpha Vantage, plus a deterministic mock for tests). Notifications are delivered today through either a Telegram bot or a logging sink, depending on the active Spring profile.

The whole system runs as a **single Spring Boot deployable** with an embedded relational option (JPA / MySQL or H2) and a MongoDB option for the persistence side. There are no microservices. There is no message broker.

---

2. Business capabilities (~ candidate bounded contexts)

Four business capabilities have been identified, each promoted to a Spring Modulith application module:

Capability	Modulith module	Core types
Portfolio management & FIFO trading	`portfolios`	`Portfolio`, `Lot`, `Transaction`
Market data acquisition	`marketdata`	`Ticker`, `StockPrice`, `MarketDataPort`

Capability	Modulith module	Core types
Watchlists & alert detection	`watchlists`	`Watchlist`, `AlertEntry`, `MarketSentinelService`
Outbound notifications	`notifications`	`NotificationChannel`, `NotificationDestination`, `NotificationSender`

The provenance of those modules is the inventory in `doc/architecture/MODULITH-BOUNDED-CONTEXT-INVENTORY.md`; their boundaries are enforced at build time by `ModulithVerificationTest`.

---

3. The four architectural styles in play, and how they coexist

```
DDD                --
>  decides where the boundaries between capabilities
```

```
Hexagonal          --
>  decides where the boundaries between layers lie
Spring Modulith    --
>  enforces both at build time
Domain Events      --
>  let one capability react to another without coupl
```

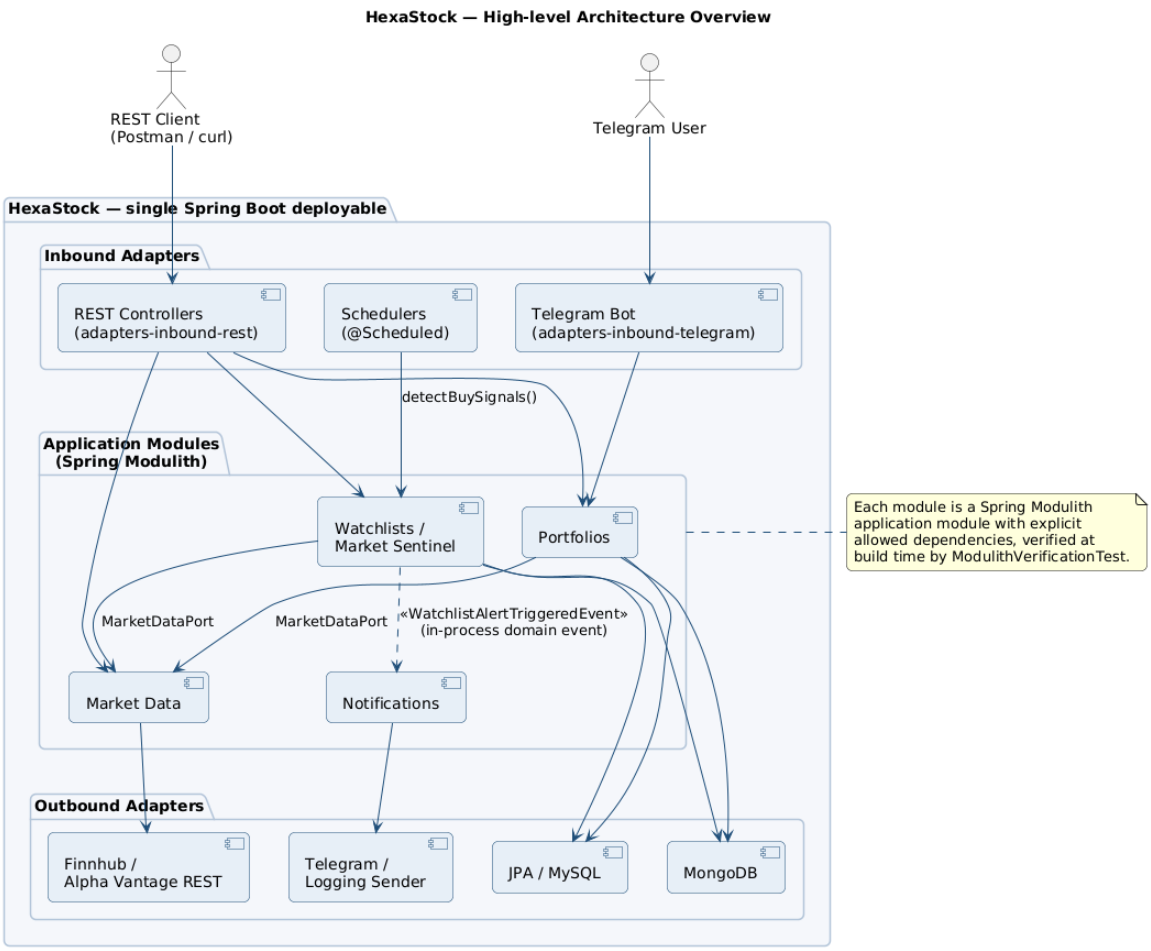


Figure 1: High-level architecture overview

3.1 DDD – strategic and tactical

- The four bounded contexts above are *strategic DDD* applied to this codebase.
- Inside each context the *tactical DDD* vocabulary is used: aggregate roots (`Portfolio`, `Watchlist`), value objects (`Money`, `Ticker`, `Price`, `ShareQuantity`), domain services where they earn their keep, repositories hidden behind ports.
- The ubiquitous language is documented in `doc/tutorial/sellStocks/UBIQUITOUS-LANGUAGE.md` and referenced by `SELL-STOCK-DOMAIN-TUTORIAL.md`.

3.2 Hexagonal Architecture – ports and adapters

- The hexagon is *per bounded context*, not global. Each capability has its own primary ports (`...UseCase`) and secondary ports (`...Port`).
- Layering is enforced by **Maven module dependencies**: `application` depends on `domain`; every adapter module depends on `application`; only `bootstrap` depends on adapters. The dependency graph is in diagrams/02-maven-multimodule.puml.
- The *application* Maven module is intentionally Spring-free (ADR-007). The

only Spring annotation tolerated there is `jakarta.transaction.@Transactional`, which is a JTA spec annotation rather than a Spring one. Everything Spring-aware lives in `bootstrap` or in outbound adapter modules.

- A second build-time guard, `HexagonalArchitectureTest` (ArchUnit), fails the build if a domain class imports `org.springframework.*`, `jakarta.persistence.*`, or `com.fasterxml.jackson.*`.

3.3 Spring Modulith – runtime enforcement of bounded contexts

- Each business capability is a top-level Java package directly under `cat.gencat.agaur.hexastock.*` – `portfolios`, `marketdata`, `watchlists`, `notifications`. That makes them detectable as Modulith application modules.

- The `@org.springframework.modulith.ApplicationMod` annotations live on `package-info.java` files inside the **bootstrap** Maven module so the `application` module stays Spring-free; runtime detection works because Spring Modulith scans by package, not by JAR.

- Allowed cross-module dependencies are declared explicitly:

```
portfolios     -> marketdata::model,
marketdata::port-out
watchlists     -> marketdata::model
notifications  -> watchlists,
marketdata::model
```

- `ModulithVerificationTest.verifyModuleStructure()` calls `MODULES.verify()` to fail the build on any cycle or undeclared cross-module dependency.

3.4 Domain Events – the only intentional decoupling mechanism today

- Exactly one domain event flows in production: `WatchlistAlertTriggeredEvent`, emitted by `MarketSentinelService` and consumed by `WatchlistAlertNotificationListener` (in the `notifications` module) using `@ApplicationModuleListener`.
- The event is a plain Java `record`. The publisher is an outbound port (`DomainEventPublisher`) implemented by `SpringDomainEventPublisher`,

which delegates to Spring's `ApplicationEvent-Publisher`. Application code never imports anything from Spring.
- See the deep dive: 02-WATCHLISTS-EVENT-FLOW-DEEP-DIVE.md.

---

4. Why this combination makes sense *here*

HexaStock is small enough that microservices would be massive over-engineering, but it is structured enough that a "fat Spring Boot project" would erode quickly. The combination of the four styles addresses that trade-off:

- **DDD gives the boundaries.** Without them every other tool degenerates into ceremony.
- **Hexagonal gives the layering.** It keeps the domain free of frameworks and makes adapters interchangeable (JPA *and* MongoDB persistence both compile against the same ports).
- **Spring Modulith gives the enforcement.** It is the difference between a bounded context being a *team agreement* and being a *failing build*.
- **Domain events give the in-process decoupling.** They let two modules collaborate without

one importing the other's internals – a property that becomes very valuable on day one of every cross-team change.

The thesis the consultancy will defend is that this combination is **deliberately conservative**: the project is one deployable, no broker, no outbox, no event store. The architectural shape, however, is the one that lets the project grow into any of those choices later without rewriting the domain.

---

5. What this overview is *not*

- It is not a defence of the current layout against the alternatives – that is in 03-LAYOUT-ALTERNATIVES.md.
- It is not a deep dive on any single mechanism – see the dedicated documents.
- It does not claim that the current implementation is production-ready under arbitrary load or reliability targets – see 04-PRODUCTION-EVOLUTION.md.

01 – Filesystem, Maven and Modulith Structure

> **Audience.** Engineers who will navigate the codebase live during the session. **Reading time.** ~15 minutes.

---

1. The three orthogonal axes

A frequent point of confusion when senior engineers first read this codebase is that it is organised along **three orthogonal axes** simultaneously:

Axis	Encoded by	Enforced by
Hexagonal layer (domain / application / adapter / bootstrap)	Maven modules	Maven dependency graph + `HexagonalArchitectureTest` (ArchUnit)

Axis	Encoded by	Enforced by
Bounded context (portfolios / marketdata / watchlists / notifications)	Top-level Java packages under `cat.gencat.agaur.hexastock.*`	`ModulithVerificationT` (`MODULES.verify()`)
Adapter technology (JPA, Mongo, REST, Telegram, market provider)	Distinct Maven adapter modules	Maven module isolation

Once you internalise that those three axes are independent, the layout reads trivially.

---

2. Maven multi-module layout

The aggregator `pom.xml` declares nine modules:

```
domain
application
adapters-inbound-rest
adapters-inbound-telegram
```

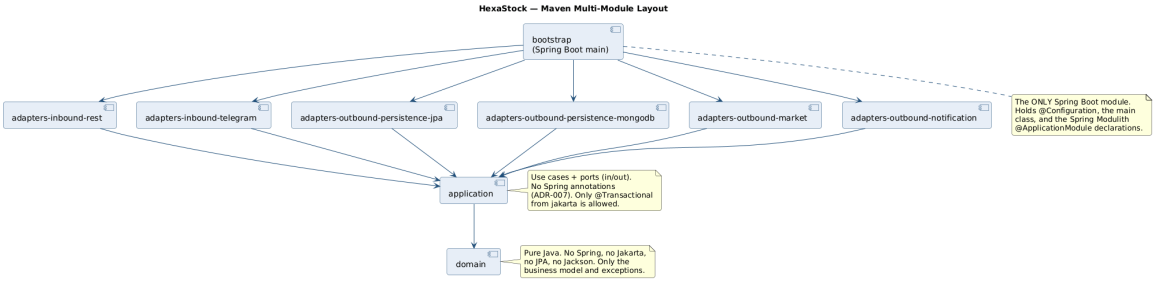


Figure 2: Maven multi-module layout

```
adapters-outbound-notification
adapters-outbound-persistence-jpa
adapters-outbound-persistence-mongodb
adapters-outbound-market
bootstrap
```

Their dependency rules are simple and strictly enforced by Maven:

- `application` depends on `domain` (and on nothing Spring).
- Every `adapters-*` module depends on `application` (and may depend on its technology stack: JPA, Spring Data MongoDB, OkHttp, Telegram Bots, etc.).
- `bootstrap` is the *only* Spring Boot module. It depends on every adapter and contains `HexaStockApplication` (`@SpringBootApplication`), `@Configuration` classes, and the Spring

Modulith `@ApplicationModule` declarations on `package-info.java`.

This forces a constraint that is easy to verify by reading any POM: the domain module's POM is *minimal* – JUnit and AssertJ for tests, nothing else. The application module's POM adds only what is strictly needed for use cases.

Why split adapters into separate Maven modules at all?

Three reasons that survived several rounds of internal review:

1. **Compile-time isolation of optional adapters.** A deployment that does not want MongoDB simply does not include `adapters-outbound-persistence-mongodb` on the classpath. The application code does not change.
2. **Forced ports first.** A new adapter cannot be written until a port exists, because the adapter module depends on `application`, not the other way round.
3. **Clear cost of leaks.** An accidental import of, say, JPA in the domain would not even compile (the domain module has no `jakarta.persistence` dependency on its classpath).

3. Filesystem layout

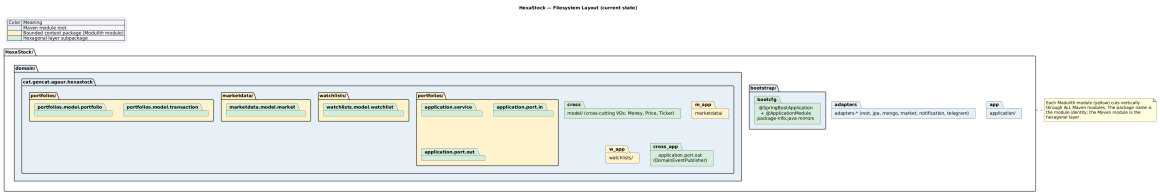


Figure 3: Filesystem layout

Inside each Maven module, the source tree follows a single rule:

```
<maven-module>/src/main/java/
  cat/gencat/agaur/hexastock/
    <bounded-context>/
      <hexagonal-sublayer>/
        ... Java types ...
```

Concrete examples for the **Watchlists** context:

```
domain/src/main/java/cat/gencat/agaur/hexastock/
  watchlists/model/watchlist/Watchlist.java
  watchlists/model/watchlist/WatchlistId.java
  watchlists/model/watchlist/AlertEntry.java

watchlists/model/watchlist/AlertNotFoundException.java

watchlists/model/watchlist/DuplicateAlertException.java
```

```
⌋
application/src/main/java/cat/gencat/agaur/hexastock/
  watchlists/package-info.java
<- javadoc + Modulith intent
  watchlists/WatchlistAlertTriggeredEvent.java
<- published API (record)

watchlists/application/port/in/WatchlistUseCase.java

watchlists/application/port/in/MarketSentinelUseCase.java

watchlists/application/port/out/WatchlistPort.java

watchlists/application/port/out/WatchlistQueryPort.java

watchlists/application/port/out/TriggeredAlertView.java

watchlists/application/service/WatchlistService.java

watchlists/application/service/MarketSentinelService.java


⌋
adapters-inbound-rest/src/main/java/cat/gencat/agaur/hexa
  watchlists/adapter/in/...
<- REST controller, DTOs


⌋
adapters-outbound-persistence-jpa/src/main/java/cat/genca
```

```
  watchlists/adapter/out/persistence/jpa/...
<- JPA entities + repository


⌋
adapters-outbound-persistence-mongodb/src/main/java/cat/g
  watchlists/adapter/out/persistence/mongodb/...
<- Mongo documents + repository


⌋
bootstrap/src/main/java/cat/gencat/agaur/hexastock/
  watchlists/package-info.java
<- @ApplicationModule lives here
```

Read that vertical slice once and you have read every bounded context – they all follow the same template.

---

4. The relationship between Maven modules, Java packages, and Modulith modules

Concept	Identity carrier	Example
Maven module	`pom.xml` artifact id	`application`

Concept	Identity carrier	Example
Hexagonal sublayer	Suffix in package name	`application.port.in`, `application.service`
Bounded context	Top-level package under `cat.gencat.agaur.hexastock`	`watchlists`
Modulith module	Same top-level package	`watchlists` (detected by Spring Modulith)
Modulith *named interface*	Inner package marked with `@NamedInterface`	`marketdata::port-out`

Three observations matter for the consultancy:

- **A Modulith module is a package, not a JAR.** It cuts vertically through Maven modules. The Watchlists Modulith module spans `domain`, `application`, `adapters-inbound-rest`, `adapters-outbound-persistence-jpa`, `adapters-outbound-persistence-mongodb`, and `bootstrap`.
- **`package-info.java` files appear twice for the same package** – once in `application` (Spring-free,

javadoc only) and once in `bootstrap` (carrying `@ApplicationModule`). This is the price ADR-007 pays to keep `application` pure. The duplication is intentional and has a long comment in the bootstrap-side `package-info.java` explaining why.

- **Cross-module dependencies are *opt-in*, not *opt-out*.** `notifications` declares `allowedDependencies = {"watchlists", "marketdata::model"}`; a fifth module appearing in its imports would fail `MODULES.verify()`.

---

5. The Spring Modulith view

Live inspection commands for the demo:

```
# Renders Modulith's own PlantUML / AsciiDoc into target/spring-modulith-docs
./mvnw -pl bootstrap test -Dtest=ModulithVerificationTest#writeDocumentation

# Verifies the dependency graph is acyclic and respects allowed dependencies
./mvnw -pl bootstrap test -Dtest=ModulithVerificationTest
```

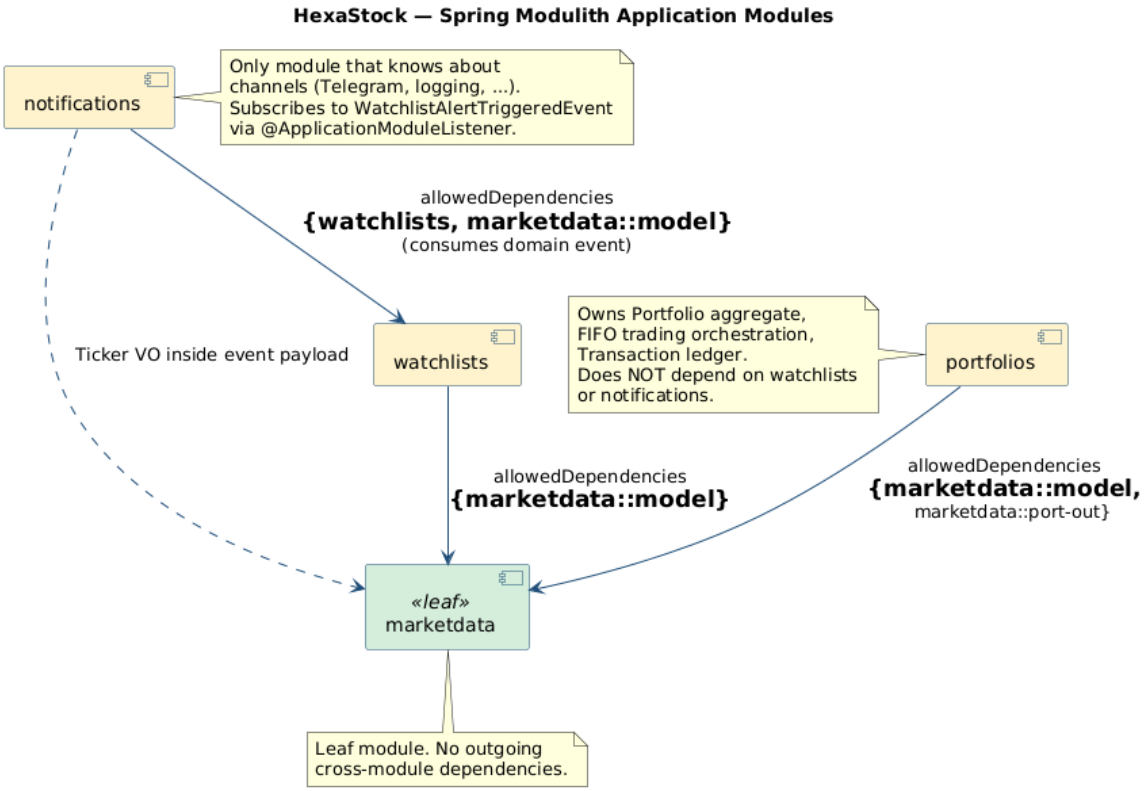


Figure 4: Spring Modulith application modules

What `MODULES.verify()` would catch the moment someone broke the rules:

- An accidental `import cat.gencat.agaur.hexastock.notifications.*` from inside `portfolios` – refused as an undeclared module dependency.
- A circular dependency between two modules – refused as a cycle.
- Any class outside `marketdata` that imports a non-`@NamedInterface` package of `marketdata` – refused as a leak past the published interface.

---

6. How the layout supports – and constrains – DDD and Hexagonal

Supports

- **Vertical slices stay coherent.** Everything that belongs to the *Watchlists* business capability lives under packages whose first segment is `watchlists`, regardless of which Maven module hosts the file.
- **Layer purity is mechanical, not aspirational.** A reviewer does not need to *spot* a Spring import in the domain – Maven would have failed the build long before review.
- **Adapters are interchangeable.** `WatchlistJpaRepositoryAdapter` and `WatchlistMongoRepositoryAdapter` both implement the same `WatchlistPort`. Swapping persistence is a Spring bean wiring choice, not a refactoring exercise.

Constrains

- **Newcomers see nine Maven modules and one tree per module – that is a lot.** The first half-day is spent learning the map. The investment pays back quickly, but it is real.
- **A single bounded context spans many directories.** Renaming the *Watchlists* module touches eight `src/main/java/cat/gencat/agaur/hexastock/watchl` trees. Modern IDEs (IntelliJ "Refactor -> Move package") handle this, but command-line `git mv` does not.
- **Two `package-info.java` files for the same package.** Discussed above – ADR-007 is the trade-off.

The competing layouts and how they compare are analysed in 03-LAYOUT-ALTERNATIVES.md.

02 – Watchlists / Market Sentinel: Domain Event Deep Dive

Audience. Engineers who will live-trace the canonical event flow during the demo. **Reading time.** ~20 minutes. **Source artefacts.** `MarketSentinelService` · `WatchlistAlertTriggeredEvent` · `DomainEventPublisher` · `SpringDomainEventPublisher` · `WatchlistAlertNotificationListener` · `NotificationsEventFlowIntegrationTest`.

---

1. Business intent

A user maintains one or more **Watchlists**. Each `Watchlist` is an aggregate that holds a list of `AlertEntry` value objects, each pinning a `Ticker` to a `Money` threshold (for example: "tell me when AAPL falls below 170 USD"). The Market Sentinel periodically scans the universe of distinct tickers across all **active** watchlists, fetches their current price from the market data provider, and – for each alert

whose threshold has been reached – publishes a domain event. The user is then notified through whichever channels the `Notifications` module knows about (Telegram or logging today; email, WebSocket, mobile push tomorrow).

The point of using a domain event here is *not* convenience: it is that the `Watchlists` module must remain **completely ignorant** of how those alerts become user-visible signals. That is the entire teaching value of the example.

---

2. End-to-end sequence

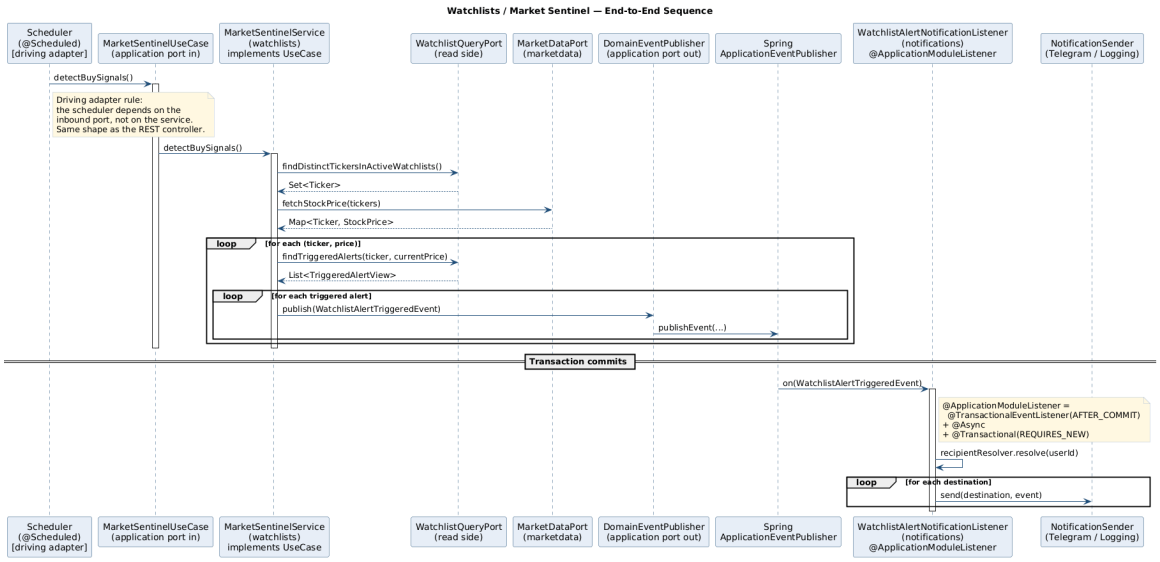


Figure 5: Watchlists / Market Sentinel sequence

The same hexagonal entry-point rule applies to the synchronous REST side. Diagram 12 shows a representative

command (`POST /api/watchlists/{id}/alerts`) flowing through the **same kind of inbound port** the scheduler uses:

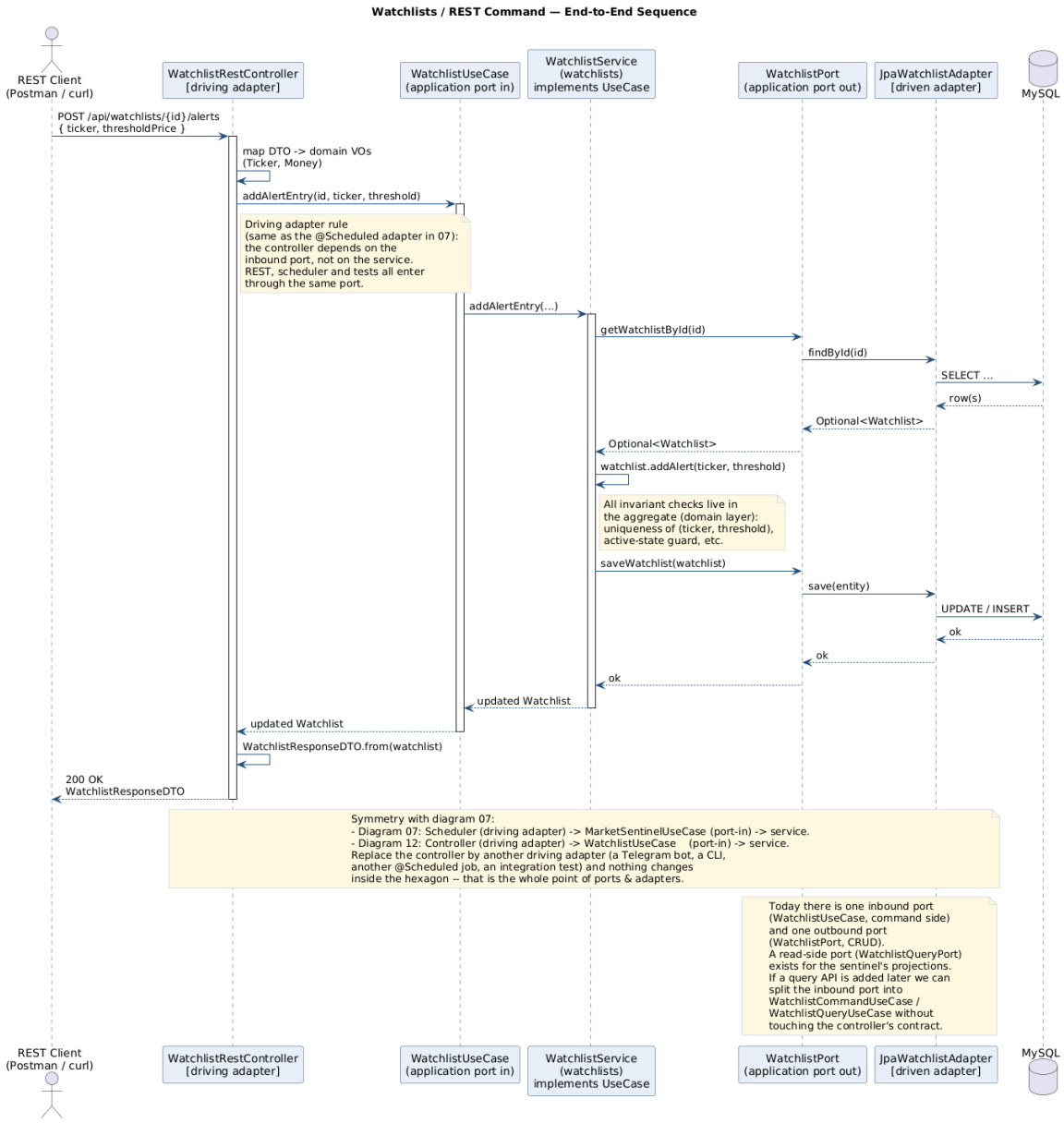


Figure 6: Watchlists REST command sequence

Both pictures share a single rule: every driving adapter (REST controller, `@Scheduled` job, Telegram bot, integration test) talks to a `*UseCase` inbound port; the applica-

tion service implements the port; the service in turn talks to the domain aggregate and to outbound ports. The two diagrams are deliberately drawn with the same vocabulary so the symmetry is visible.

The flow is short enough to walk verbally during the demo:

1. A driving adapter – the `MarketSentinelScheduler` (`@Scheduled`) or an integration test – calls the **inbound port** `MarketSentinelUseCase.detectBuySignals()`. The scheduler does **not** know `MarketSentinelService` exists; that is the whole point of hexagonal – every entry point (REST, scheduler, test) goes through a port, and the port is implemented by the application service.
2. The service implementation reads, on a CQRS-style **read port** (`WatchlistQueryPort.findDistinctTickersInActiveW` the set of tickers currently being watched.
3. It calls `MarketDataPort.fetchStockPrice(tickers)`, returning a `Map<Ticker, StockPrice>`. This is the only outgoing call that crosses process boundaries.
4. For each price, it asks the read port `findTriggeredAlerts(ticker, price)` for the

materialised list of `TriggeredAlertView` projections.

5. For each triggered alert it builds a `WatchlistAlertTriggeredEvent` and calls `DomainEventPublisher.publish(event)`.

The publication call returns immediately; the listener does not run inside the calling transaction (see §5).

---

3. The event itself – anatomy of a good domain event

```
public record WatchlistAlertTriggeredEvent(
        String watchlistId,
        String userId,
        Ticker ticker,
        AlertType alertType,
        Money threshold,
        Money currentPrice,
        Instant occurredOn,
        String message
) {
    public WatchlistAlertTriggeredEvent { /*
    explicit non-null validation */ }
```

```
    public Optional<String> messageOptional() {
    ... }
    public enum AlertType {
    PRICE_THRESHOLD_REACHED }
}
```

Five properties are worth highlighting because they recur in every healthy domain-event design:

Property	Embodied here by
It is a Java `record`. Immutable, value-equal, no setters.	The whole class.
It carries business identity, not infrastructure identity.	`userId` is the watchlist owner, *not* a Telegram chat id, *not* an email address. The `notifications` module resolves channels later.
It validates its invariants at construction.	The compact constructor calls `Objects.requireNonNull` on every required field.
It is timestamped at the source.	`occurredOn` is set by the publisher's injected `Clock`, never inferred downstream.

Property	Embodied here by
It is enum-typed, not string-typed.	`AlertType.PRICE_THRESHOLD_REACHED` keeps the contract evolvable without breaking existing consumers.

What it deliberately does **not** carry: any mention of channels, transport, retries, or downstream system identities. Those would couple the publisher to the consumer.

---

4. Where the event lives – and why that matters

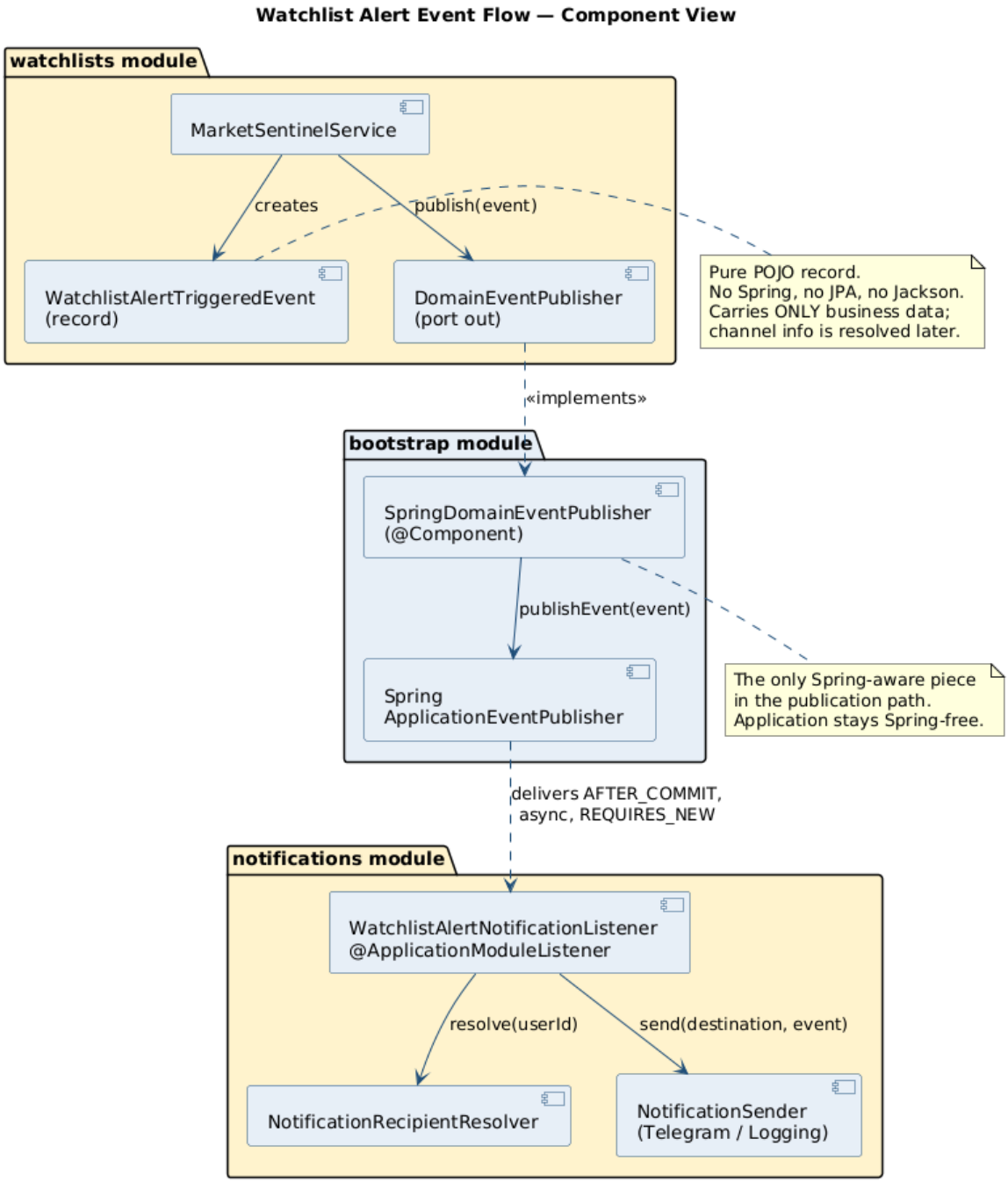


Figure 7: Component view

Concern	Where	Why
Event type	`application` Maven module, package `cat.gencat.agaur.hexastock.watchlists`	The published API of the Watchlists module. Living in the application module makes it visible to consumers without dragging the domain along, and keeps it Spring-free.
Publisher port	`application/.../application/port/out/`	The Java interface, the only contract the application service knows.
Publisher adapter	`bootstrap/.../configuration/SpringDom`	Bridges the port to Spring's `ApplicationEventPublisher`. The only Spring-aware piece.

Concern	Where	Why
Consumer	`adapters-outbound-notification/...WatchlistAlertNotifi`	Lives in the `notifications` module. Spring's `@ApplicationModuleListener` does the wiring.

The bidirectional rule is: **publishers know the event, consumers know the event, neither knows the other.** That property is what lets you add a second consumer (an `AuditTrailListener`, say) without touching the publisher.

---

5. Transactional semantics – the real reason `@ApplicationModuleListener` exists

```
@ApplicationModuleListener
public void on(WatchlistAlertTriggeredEvent event)
{ ... }
```

`@ApplicationModuleListener` is a Spring Modulith convenience that expands to:

```
@TransactionalEventListener(phase = AFTER_COMMIT)
@Async
@Transactional(propagation = REQUIRES_NEW)
```

Each piece earns its place:

- **`AFTER_COMMIT`** – guarantees the publisher's transaction has *successfully committed* before the listener runs. If the publisher rolls back, the event is silently discarded. This is the entire reason this annotation exists in Modulith: it makes "I detected a triggered alert and I want to notify the user" a transactionally honest assertion.
- **`@Async`** – runs the listener on a separate thread. The publishing transaction is not slowed down by Telegram latency or by a slow recipient resolver.
- **`@Transactional(REQUIRES_NEW)`** – gives the listener its own transactional context. If a future consumer needs to write to the database (an audit log, for example), it writes in *its own* transaction; a failure does not roll back the publishing transaction (which is already committed) and does not cascade to other listeners.

These three properties together are what make in-process domain events safe enough to be used in a real codebase rather than being a leaky abstraction.

---

6. The notification side – why it can be ignored from the publisher's point of view

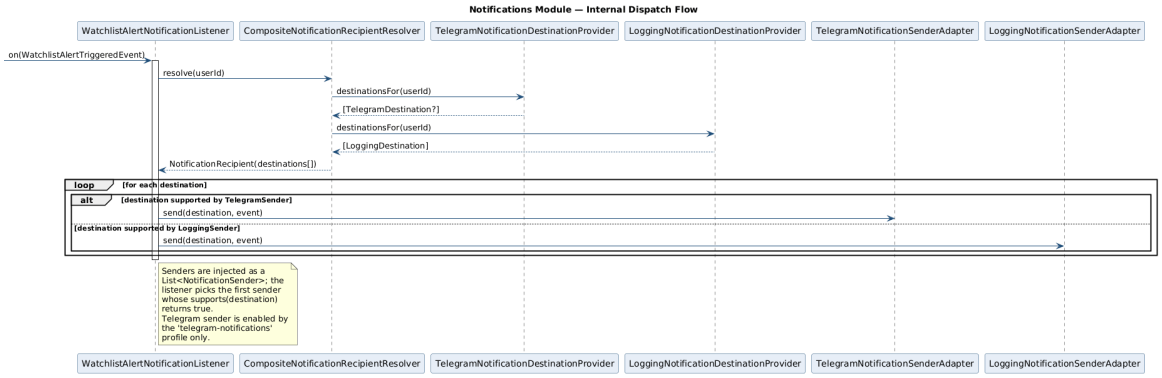


Figure 8: Notification dispatch flow

Inside the listener:

1. `NotificationRecipientResolver.resolve(userId)` produces a `NotificationRecipient` carrying *every* known destination for that user. The default implementation (`CompositeNotificationRecipientResolver`) aggregates results from each `NotificationDestinationProvider` bean – today: a Telegram

one (active under the `telegram-notifications` profile) and a logging one (always active).
2. For each `NotificationDestination`, the listener picks the first `NotificationSender` whose `supports(destination)` returns true, and calls `send(destination, event)`.

That entire mechanism is private to the `notifications` module. The `watchlists` module has no idea any of those types exist. Adding SMS would be:

- a new `SmsNotificationDestination`,
- a new `SmsNotificationDestinationProvider` (resolves a phone number for a `userId`),
- a new `SmsNotificationSenderAdapter`.

No change to `Watchlists`. No change to the event. No change to the listener. That is what the consultancy is selling.

---

7. Why this is a perfect teaching example

A short, defensible list:

- It is **real code that runs**, not an abstract example. The flow is exercised by `NotificationsEventFlowIntegrationTest` and (in deployment) by a `@Scheduled` job.
- It exercises **all four architectural styles at once** – DDD (the `Watchlist` aggregate), Hexagonal (the `DomainEventPublisher` port + Spring adapter), Modulith (the `@ApplicationModule` boundary and `@ApplicationModuleListener` consumer), and Domain Events (the `WatchlistAlertTriggeredEvent` record).
- It demonstrates **asymmetric coupling**: `notifications` declares `allowedDependencies = {"watchlists", "marketdata::model"}`, but the inverse is *not* declared and would fail the build. That asymmetry is the entire point.
- It demonstrates the **CQRS read side** without going full CQRS: `WatchlistQueryPort` returns flat `TriggeredAlertView` projections, distinct from the `Watchlist` aggregate **write path** (which loads the full aggregate in order to mutate it). The sentinel never loads the aggregate; the controller only loads it on write commands.
- The behaviour is **safely reversible** – disabling the listener (commenting out `@ApplicationModuleListener`) produces zero alerts, but breaks no

invariants. That property would not hold if `MarketSentinelService` were calling `notificationSender.send(...)` directly.

---

8. Caveats and honest limitations

- The event is **in-memory only**. If the JVM crashes between `AFTER_COMMIT` and the listener body running, the event is lost. There is no outbox today. See 04-PRODUCTION-EVOLUTION.md §2 for what to do about this.
- The listener's `@Async` execution uses Spring's default executor. A production deployment should configure a bounded executor with explicit rejection policy.
- There is no retry. A `Sender.send(...)` failure is logged and dropped. In production, retries would belong in the sender adapter (or in the broker layer once the event is externalised), *not* in the listener.
- The `notifications` module currently depends on the `Ticker` value object from `marketdata`, which it includes in the rendered message. That keeps the example honest but means a hypothetical extraction would still need a shared kernel for `Ticker`. A future

refactor could carry the ticker as a `String` in the event instead, eliminating that dependency.

03 – Layout Alternatives: a Rigorous Comparison

> **Audience.** Senior engineers who will, correctly, ask *"why did you split it like this rather than like that?"*. **Reading time.** ~20 minutes.

This document compares the current HexaStock layout against four common alternatives for projects that combine **Spring Modulith + DDD + Hexagonal Architecture + bounded-context thinking**, and explains the position the project takes.

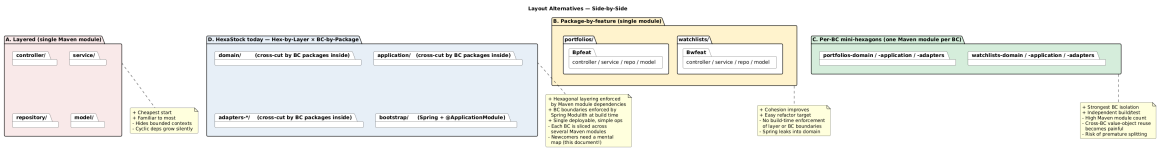


Figure 9: Layout alternatives at a glance

A. Layered, single Maven module

Shape. One Maven artefact. Top-level packages by *technical layer* – `controller`, `service`, `repository`, `model`.

The bounded contexts are not visible at the package level; they are diluted across the four layer packages.

Strengths. - The shape every Spring tutorial uses; new joiners need zero ramp-up. - Cheapest possible build; everything compiles in one go. - Refactoring across layers is trivial because nothing is enforced.

Weaknesses. - Bounded contexts are *invisible*. A `WatchlistService` and a `PortfolioService` end up next to each other in the same `service` package, and a junior developer will gladly inject one into the other. - There is no build-time guarantee that the domain is framework-free. - Spring Modulith has nothing to enforce – everything lives in the same module. - Cycles grow silently. By the time `controller -> service -> repository -> service` cycles appear, unwinding them is a multi-week project.

Good fit for. A throwaway prototype, a take-home interview project, or a codebase whose lifetime is measured in months.

Comparison with HexaStock. This is the layout HexaStock would degenerate into if every guard rail (`HexagonalArchitectureTest`, `ModulithVerificationTest`, the multi-module Maven graph) were removed. The project's thesis is that the incremental cost

of avoiding this layout is small and pays back almost immediately.

B. Package-by-feature, single Maven module

Shape. One Maven artefact. Top-level packages by *bounded context* – `portfolios/`, `watchlists/`. Inside each, sub-packages mirror the layered shape – `portfolios/controller`, `portfolios/service`, `portfolios/repository`, `portfolios/model`.

Strengths. - Bounded contexts become visible the moment you open the project tree. - A reviewer can see a feature-shaped commit; cross-context noise drops. - It is the natural starting point for an *incremental* migration toward Modulith – it is exactly what was done in the early stages of the HexaStock refactoring program (see SPRING-MODULITH-GLOBAL-REFACTORING-PLAN.md).

Weaknesses. - Layer boundaries are not enforced; nothing prevents a controller importing a repository directly. Spring Modulith does not catch this – it polices *modules*, not *layers*. - Bounded-context boundaries are not enforced

either, unless Spring Modulith is added on top with explicit `allowedDependencies`. Without that, a `WatchlistsService` can still import a `PortfoliosService` freely. - The domain is on the same classpath as Spring; the "no Spring in the domain" rule is aspirational only.

Good fit for. Single-team projects of moderate size where the team can be trusted with conventions but not with build-time enforcement.

Comparison with HexaStock. Adding `spring-modulith-core` to a layout B project gets you most of the *bounded-context* enforcement that HexaStock has, but none of the *layer* enforcement. The HexaStock thesis is that paying the multi-Maven-module tax buys you the latter.

C. Per-bounded-context mini-hexagons (Maven module per BC)

Shape. One Maven artefact per bounded context, each containing its own `-domain`, `-application`, `-adapters` sub-modules. So: `portfolios-domain`,

`portfolios-application`, `portfolios-adapters-jpa`, `portfolios-adapters-rest`, etc. – multiplied by the number of contexts.

Strengths. - The strongest possible isolation. Each BC is a fully independent hexagonal application; it could in principle be extracted to its own repository tomorrow. - Build-time impossible to import across BCs unless the dependency is declared in the consumer's POM. - Independent test execution per BC.

Weaknesses. - The Maven module count explodes. With four bounded contexts and three hexagonal layers each, you get ~12 modules before any persistence variant is added, and ~20 once you split JPA/Mongo/REST adapters per BC. - **Cross-BC value-object reuse becomes painful.** `Money`, `Ticker`, `Price` need to live in a shared kernel module that every BC depends on, which becomes a magnet for "I'll just add this here for now". - High risk of *premature splitting*. Bounded contexts are not always obvious early; this layout makes them very expensive to merge later. - Setup, IDE indexing, and CI friction are non-trivial.

Good fit for. Multi-team projects where each BC has a separate codeowner and the BC boundaries have already been validated by months of operational experience. Often a stop on the way to physically separated services.

Comparison with HexaStock. This is what HexaStock might evolve into *if* the project grows to multiple teams, *if* one BC starts shipping at a materially different cadence than the others, and *if* the cost of the extra Maven modules is justified by the team structure. None of those is true today.

D. Modulith-first (top-level package per module, single Maven module)

Shape. One Maven artefact. Top-level packages directly map to Spring Modulith application modules. Internal sub-packages can use any internal convention. Everything is enforced by Spring Modulith only.

Strengths. - This is the canonical Spring Modulith reference shape (the `kitchensink` sample). - Lowest ceremony for a Modulith-first design. - Build-time BC isolation through `MODULES.verify()`.

Weaknesses. - Hexagonal layering inside a module is *convention*, not *enforcement*. - Domain code can import

Spring; no compile-time wall protects it. - A purely package-based hexagon needs ArchUnit rules to be more than a visual organisation choice.

Good fit for. Greenfield projects that want Modulith's BC enforcement *now* and are happy to layer hexagonal discipline on top via ArchUnit rather than via Maven modules.

Comparison with HexaStock. HexaStock is essentially "Layout D plus hexagonal layering enforced by Maven modules". The extra Maven modules pay for the layering enforcement; the top-level package convention pays for the BC enforcement.

E. HexaStock today – Hexagonal × Bounded Context, by Maven *and* by package

Shape. Already described in 01-FILESYSTEM-AND-MAVEN-STRUCTURE.md:

- *Hexagonal layer* -> Maven module (`domain`, `application`, `adapters-*`, `bootstrap`).

- *Bounded context* -> top-level Java package (`portfolios`, `marketdata`, `watchlists`, `notifications`).
- *Hexagonal sublayer inside a BC* -> Java sub-package (`watchlists.application.port.in`, `watchlists.application.service`, `watchlists.adapter.out.persistence.jpa`).

Strengths. - Hexagonal layering is *mechanically* enforced by the Maven dependency graph: the `application` module's POM does not contain Spring; the `domain` module's POM does not even contain Jakarta. A leak does not compile. - Bounded contexts are *mechanically* enforced by Spring Modulith via `MODULES.verify()`; cross-module imports must be declared in `allowedDependencies`. - Domain events stay framework-free: they live in `application` (a Spring-free Maven module) and only the adapter (`SpringDomainEventPublisher`) knows about Spring. - Persistence variants (JPA *and* MongoDB) coexist as alternative Maven adapter modules selecting at deployment time.

Weaknesses. - Each BC is split across many Maven modules; an IDE-driven mental map is needed. - `package-info.java` files appear twice for the same package (Spring-free copy in `application`, Spring-aware copy with `@ApplicationModule` in `bootstrap`). The duplication is in-

tentional but is the kind of thing senior engineers will rightly question. - Newcomers spend the first half-day learning the layout. The investment pays back quickly, but it is real.

When this is right. When you want the *enforcement* of layered hexagonal *and* the enforcement of bounded contexts, *and* the project is small enough that one Spring Boot deployable still makes operational sense. That is precisely HexaStock today.

How to explain this coherently to senior engineers

A four-line summary that survives any whiteboard:

> "We picked the layout that lets the build itself enforce two orthogonal things – *layer purity* via Maven modules, and *bounded-context isolation* via Spring Modulith. The cost is one extra `package-info.java` per module and a slightly larger Maven graph. The benefit is that we will never *accidentally* leak Spring into the

domain or `WatchlistsService` into `Portfolios`, and we paid that cost on day one rather than during the first cross-team incident."

Then, if pushed:

- Layout A would let the project ship faster *today*, and erode faster *next quarter*.
- Layout B is what HexaStock looked like during early Modulith phases; it was the right *transitional* shape, not the right *destination*.
- Layout C is what HexaStock might *eventually* become, if and only if the team structure and the cadence pressure justify the multiplication of Maven modules.
- Layout D is what HexaStock would look like without the Maven module split – it would be cleaner to navigate, but `domain/` would not be guaranteed framework-free.

The current shape is a deliberate, conservative bet on *enforcement over ergonomics* – a bet that pays off as soon as the codebase is touched by more than one engineer.

04 – From Current Solution to Realistic Production Evolution

> **Audience.** Engineers and architects who will ask *"and what would you do at 10× the load?"*. **Reading time.** ~15 minutes.

The current Watchlists / Notifications event flow is intentionally simple: in-memory, in-process, no broker, no outbox. This document explains what that buys you, where it stops being safe, and the realistic evolution paths – none of which require throwing away the current architecture.

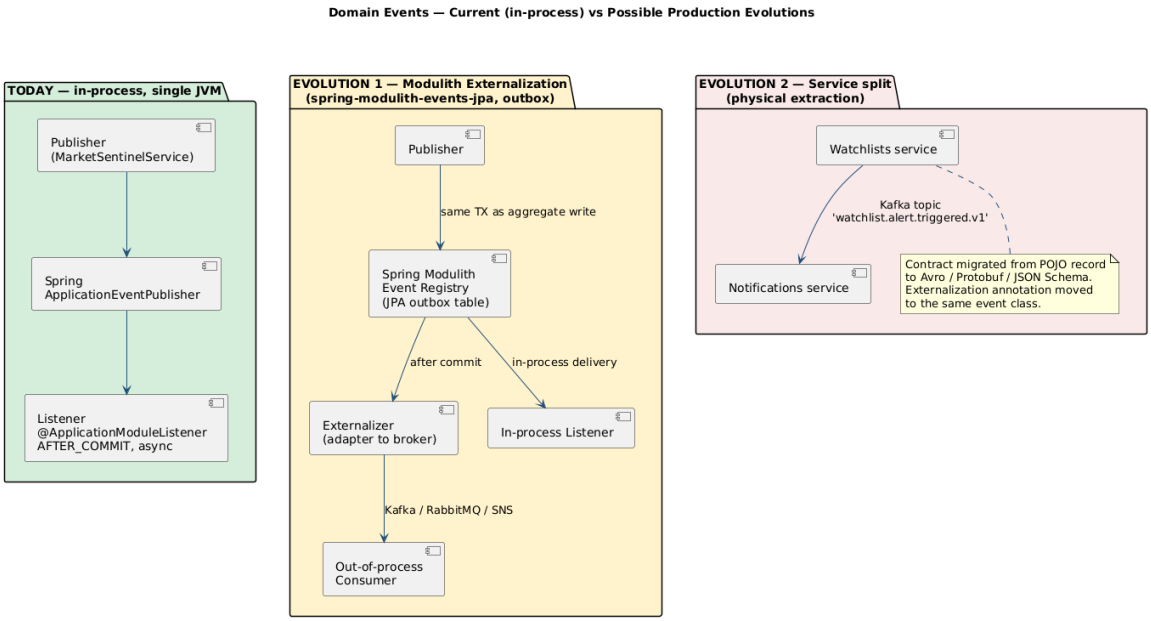


Figure 10: Current vs future event topologies

1. What the current implementation actually guarantees

Property	Today	How
Event is delivered iff the publishing transaction commits	[OK]	`@ApplicationModuleLis` => `@Transactional EventListener(AFTER_COMMIT)`
Listener does not slow down the publisher	[OK]	`@ApplicationModuleLis` => `@Async`
Listener failure does not roll back the publisher	[OK]	`@ApplicationModuleLis` => `@Transactional(REQUIRES_NEW)`
Event survives a JVM crash between commit and listener invocation	[X]	No persistence; in-memory event bus
Event survives a listener exception	[X]	No retry; logged and dropped
Event has a global ordering guarantee	[X]	None requested or provided

Property	Today	How
Event is consumable from another process	[X]	In-process only
Event has a published, versioned schema	[!]	Java record contract; no machine-checkable schema

The honest summary: it is **good enough for a single-deployable, single-team financial-portfolio back-end with non-mission-critical notifications**, and is *architecturally shaped* so that any of the missing properties can be added without rewriting the publisher.

---

2. Evolution path 1 – add durability inside the same monolith

The Spring Modulith project ships `spring-modulith-events-jpa` (and equivalents for MongoDB, JDBC, Kafka, etc.). Adding it gives the **transactional outbox pattern** with effectively no code changes:

```
<dependency>

<groupId>org.springframework.modulith</groupId>
    <artifactId>spring-modulith-starter-jpa</artifactId>
</dependency>
```

What you get the moment you flip that switch:

- Every `ApplicationEventPublisher.publishEvent(event)` whose event class is marked `@org.springframework.modulith.events.Externaliz` (or annotated with the framework-level externalisation marker) is **persisted to an outbox table inside the same JDBC transaction** as the publisher's writes.
- Listener invocations become **completion records** in that same table.
- A scheduled poller / republisher retries failed listener invocations until they succeed (or are explicitly dead-lettered).

The publisher does not change. The listener does not change. What changes:

- The application has a new database table (`event_publication`), so a schema migration is needed.

- The listener should be made *idempotent*, because retries become possible. In practice this means designing the side effect to tolerate a duplicate invocation – which is good practice anyway.

The architectural diagram is `EVOLUTION 1` in the figure above.

---

3. Evolution path 2 – externalise selected events to a broker

Once the outbox is in place, **externalising** a specific event becomes a configuration change rather than a redesign. Spring Modulith supports externalising to Kafka, RabbitMQ, AWS SNS/SQS, JMS, etc., via additional starters and a single annotation on the event class:

```
@org.springframework.modulith.events.Externalized
("watchlist.alert.triggered.v1")
public record WatchlistAlertTriggeredEvent(...) {
... }
```

The framework will then publish each persisted event to the configured broker after commit, with at-least-once semantics backed by the outbox.

What this enables:

- A second consumer in **another process** (a notifications worker, a reporting job, a fraud-detection pipeline) can subscribe to the broker topic without modifying HexaStock.
- A truly async processing pipeline that survives consumer outages.
- A natural seam for a future *physical* extraction of the consumer (see §4).

What it costs:

- The event becomes a **published contract** in the strong sense. Consumers outside the codebase will rely on it. Versioning discipline becomes mandatory: name the topic `*.v1`, plan for `*.v2`, never silently change field semantics.
- The Java `record` POJO is no longer the contract – the **wire format schema** is. Avro, Protobuf, or JSON Schema (with a registry) becomes a serious choice, not a stylistic one.
- Operational concerns enter: broker availability, consumer lag, dead-letter topics, retry budgets.

---

4. Evolution path 3 – physically extract a module

If `notifications` becomes either (a) a substantially heavier workload than the rest of the application, (b) a separately-owned product, or (c) a compliance-isolated component, it can be extracted into a separate deployable. The architecture is already shaped for this:

- The publishing side already speaks an external contract (after step 3).
- The notifications module already imports nothing from `portfolios` and only consumes the published `WatchlistAlertTriggeredEvent` plus the `Ticker` VO.
- The module's adapters (`TelegramNotificationSenderAdapter`, `LoggingNotificationSenderAdapter`) are independently deployable units.

Extraction is not free, of course:

- The shared `Ticker` value object now needs a strategy: vendored copy, shared library, or replaced by a `String` ticker inside the event.
- The persistence story for the notifications side (if any) becomes a separate database, not a separate schema.

- Operational concerns (CI, deployment pipelines, observability, identity/auth between services) are now in scope.

The point: this is a *capability you preserve*, not a *target you commit to*. The current layout makes the option cheap to keep open without paying the cost upfront.

---

5. What the current architecture deliberately *does not* claim

These are the honest "no" answers to the questions a senior engineer will ask:

- **"Is this microservices-ready?"** – No, and it does not claim to be. It is *deployable-monolith-correct* and *extraction-friendly*. Those are different goals.
- **"Does this guarantee exactly-once delivery?"** – No. Exactly-once does not exist outside of constrained transactional boundaries. The current flow is *at-most-once in memory*; with the outbox it becomes *at-least-once with idempotent consumers*.

- **"Is the event a stable API?"** – Today it is an internal Java record. The moment a second team consumes it, treat it as a stable API and apply versioning rules. Not before.
- **"Should every aggregate operation publish a domain event?"** – No. Publish events for **business facts other modules legitimately want to react to**. Publishing for the sake of "decoupling" leads to event spam and makes the aggregate harder to reason about.

---

6. Where this leaves the consultancy thesis

> "We picked an architecture that is honest about today's needs and shaped for tomorrow's options. The Watchlists event flow is the smallest correct example of that thesis: in-process today, outbox-and-broker-ready tomorrow, extractable the day after – and none of those moves require us to revisit the Watchlists *domain* code."

That sentence is the take-away. Everything else in this folder is the evidence that backs it.

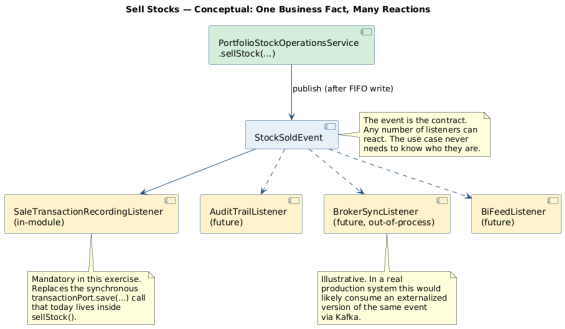
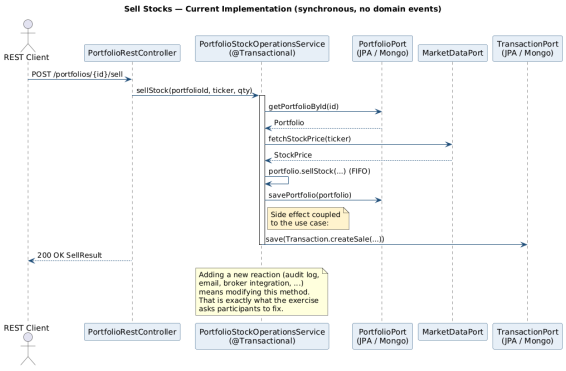
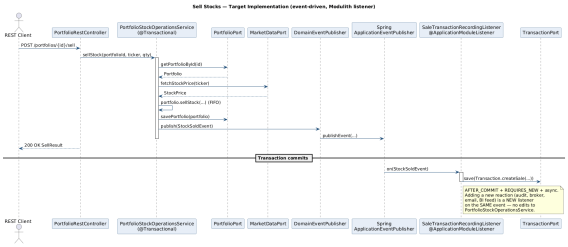
Exercise – Refactor *Sell Stocks* with Domain Events

Audience. Senior Java engineers, software architecture students and professional training participants who are already comfortable with HexaStock's hexagonal layout and the requirement-traceability model.

Format. Reference demo (instructor) + hands-on refactor (participants) + facilitated review.

Scope. Documentation only. This exercise asks participants to refactor live code; it does *not* alter the codebase as committed.

At a glance

Idea	Diagram
Conceptual – one business fact, many reactions	
Today – synchronous, two persistence side effects in one method	
Target – event-driven, listener writes the transaction	

The companion documentation set for the upcoming consultancy session lives under doc/consultancy/monday-session/.

---

1. Why this exercise exists

HexaStock already contains one fully working domain-event flow: the *Watchlist / Market Sentinel -> Notifications* pipeline. That pipeline is documented in doc/tutorial/watchlists/WATCHLISTS-MARKET-SENTINEL.md and dissected end-to-end in the consultancy briefing chapter 04-DOMAIN-EVENTS-DEEP-DIVE.md.

That flow is the *instructor demo*. It exhibits the canonical separation between a *fact* (an alert condition was reached for a ticker) and *reactions* to that fact (a notification is dispatched). The publishing module – Watchlists – knows nothing about how, when or through which channel the reaction is performed.

This exercise asks participants to apply the same separation to a different, equally important business operation: **selling stocks**. The current implementation in PortfolioStockOperationsService.sellStock(…) couples the *sell decision* with one of its consequences (recording a `Transaction`) inside a single synchronous method. The exercise is

to disentangle them using the same pattern that the Watchlists module already exemplifies.

The pedagogical intent is to give participants the experience of *recognising* the pattern in working code, *justifying* its application in a new context and *implementing* the refactor end-to-end.

---

2. The instructor demo: the existing Market Sentinel flow

Spend approximately fifteen minutes walking through the in-flight implementation. The salient artefacts are:

Layer	Artefact
Event (POJO record)	WatchlistAlertTriggeredEvent.java
Outbound port (publisher abstraction)	DomainEventPublisher.java
Spring adapter for the publisher	`bootstrap/.../config/events/S`
Publication site (application service)	MarketSentinelService.detectBuySign
Consumer (`@ApplicationModuleListener`)	WatchlistAlertNotificationListener.ja

Layer	Artefact
End-to-end integration test	NotificationsEventFlowIntegrationTe
Architectural verification	ModulithVerificationTest.java (assertions `watchlistsHasNoOutgoingModuleDependencies` and `notificationsOnlyDependsOnWatchlists`)

Five properties of this implementation should be made explicit during the demo, because the participants will be asked to reproduce them:

1. **The event is a Java `record` with no framework imports.** It compiles in the framework-free `application` Maven module (see ADR-007).
2. **The event carries business identity, not transport identity.** It contains `userId`, never `chatId` or `email`.
3. **Publication goes through a port (`DomainEventPublisher`), not Spring's `ApplicationEventPublisher` directly.** This preserves the application layer's framework independence.
4. **The consumer uses `@ApplicationModuleListener`, which is `@Transac-`**

`tionalEventListener(AFTER_COMMIT) + @Async + @Transactional(REQUIRES_NEW).` The reaction therefore runs only after the publishing transaction commits, on a different thread, in its own transaction.

5. **The publisher knows nothing about the consumer.** No `import` statement in `MarketSentinelService` references anything in the `notifications` package; the `ModulithVerificationTest` asserts this property in CI.

Make the participants *see* property 5 by opening both files side by side. Decoupling is concrete.

---

3. The current *Sell Stocks* implementation

The participants' starting point is the production code in PortfolioStockOperationsService.sellStock(…):

```
@Override
@RetryOnWriteConflict
public SellResult sellStock(PortfolioId
portfolioId, Ticker ticker, ShareQuantity
quantity) {
```

```
    Portfolio portfolio =
portfolioPort.getPortfolioById(portfolioId)
            .orElseThrow(() -> new
            PortfolioNotFoundException(
            portfolioId.value()));

    StockPrice stockPrice =
stockPriceProviderPort.fetchStockPrice(ticker);
    Price price = stockPrice.price();

    SellResult sellResult =
portfolio.sell(ticker, quantity, price);
    portfolioPort.savePortfolio(portfolio);

    Transaction transaction =
Transaction.createSale(
            portfolioId, ticker, quantity, price,
sellResult.proceeds(), sellResult.profit());
    transactionPort.save(transaction);

    return sellResult;
}
```

Three responsibilities are interleaved inside this single method:

1. **The core business operation.** Loading the portfolio, fetching the current price and invoking `port-`

`folio.sell(...)`. This is what *Sell Stocks* fundamentally *is*.

2. **State persistence.** `portfolioPort.savePortfolio(portfolio)` makes the new aggregate state durable.
3. **A side effect: recording a financial transaction.** `Transaction.createSale(...)` followed by `transactionPort.save(transaction)` writes an audit-style record into a separate `Transaction` collection / table. The `Transaction` aggregate is *not* part of the `Portfolio` aggregate; it lives under domain/…/transaction with its own port TransactionPort.java and its own persistence adapters.

Responsibility 3 is precisely what should be separated from responsibility 1. It is a *reaction* to a business fact (a sale happened), not a *constituent* of the fact itself.

---

4. The architectural problem

Three concrete problems flow from the current arrangement.

4.1 Open-closed violation under future reactions

Today the only consequence of a sale is that one `Transaction` row is recorded. Future plausible consequences include – non-exhaustively – an entry in an immutable audit ledger (Section 5.2 of the domain-events roadmap), a real-time analytics projection of trading volume, a per-tax-year realised-gain aggregator, an outbound message to a brokerage reconciliation queue, an opt-in confirmation notification to the user, and a re-evaluation of any *position-size* watchlist alert configured by the same user on the same ticker. Each new reaction means a new line in `sellStock(...)`, a new injected port and a new responsibility attributed to a service whose name still claims to be about *stock operations*.

4.2 Transactional coupling between the decision and the reaction

The transaction record is written inside the same JPA / Mongo transaction as the portfolio mutation. If a future reaction is slow, fragile or involves an external system, that fragility leaks back into the user-facing `sellStock` path. A failed audit append should not roll back a successful sale; today's design has no mechanism to express that.

4.3 Bounded-context leakage as the platform grows

A future *audit*, *reporting* or *integrations.brokerage* module will need to know that a sale has occurred. Each such module would otherwise have to call `TransactionService.getTransactions(...)` on a polling basis and reconstruct the fact from the row, or – worse – depend directly on `PortfolioStockOperationsService`. Both options re-introduce the cross-module coupling that Spring Modulith exists to prevent.

A domain event addresses all three problems with a single mechanism.

---

5. The exercise

The exercise is performed on a new feature branch, branched from the current experimental branch (or from `main`, depending on the participants' working agreement). It is performed *without modifying tests* – except to add new tests that verify the new behaviour – and *without weakening any existing architectural assertion.*

5.1 Required outcome

Participants must deliver a refactor in which:

1. The application service `PortfolioStockOperationsService.sellStock(...)` continues to return the same `SellResult` and continues to enforce the same invariants.
2. The synchronous call to `transactionPort.save(...)` is *removed* from `sellStock(...)`. (The buy path is out of scope unless participants choose to extend the refactor; see Section 8.)
3. A new domain event – name and shape to be decided by the participants, based on the project's ubiquitous language – is published from `sellStock(...)` through the existing `DomainEventPublisher` port.
4. A new `@ApplicationModuleListener` consumes the event and writes the `Transaction` record. This listener replaces the synchronous side effect with an after-commit reaction.
5. All existing tests remain green. New tests cover the new event flow at the same level of rigour as NotificationsEventFlowIntegrationTest.

6. The Modulith verifications in ModulithVerificationTest and the architecture rules in HexagonalArchitectureTest remain green.

5.2 Open design questions for the participants

The exercise is deliberately *not* prescriptive about the following points. Each participant – or each pair – is expected to take a position, justify it and defend it during the review.

- **What is the event called?** Candidates include `StockSoldEvent`, `SaleExecutedEvent`, `PortfolioStockSoldEvent` and others. The name must respect the project's ubiquitous language and match the past-tense convention used by `WatchlistAlertTriggeredEvent`.
- **What does the event carry?** At minimum: portfolio identity, owner identity, ticker, quantity sold, sale price per share, proceeds, realised profit, instant of occurrence. Whether to include a list of consumed lots – the canonical *"one aggregate operation, multiple emitted events"* extension – is the more interesting design question. See Section 7.
- **In which package does the event live?** A defensible answer is

`cat.gencat.agaur.hexastock.portfolios.events` (a new published-API sub-package), declared with `@NamedInterface("events")` in the bootstrap-side mirror `package-info.java`. Compare with how `cat.gencat.agaur.hexastock.marketdata.model.mar` is exposed (see marketdata/model/market/package-info.java).

- **Where does the listener live?** The Transaction recording is a *Portfolio Management* concern – the `Transaction` aggregate is owned by Portfolio Management – so the listener lives inside `cat.gencat.agaur.hexastock.portfolios`. This is *intra-module* event consumption, which is a perfectly valid use of `@ApplicationModuleListener` and an excellent first step before introducing cross-module consumers.
- **Does the aggregate stay value-returning, or does it accumulate events internally?** Both styles are acceptable in DDD literature. HexaStock's `Portfolio` is currently value-returning (it returns a `SellResult`), which makes Vernon's "register events as a side effect of behaviour" idiom (IDDD, Ch. 8) the *less* idiomatic choice for this codebase. Discuss with the participants which style they prefer and why.

- **What `allowedDependencies` change in the `portfolios` `@ApplicationModule`?** If the listener is intra-module, none. If a future cross-module consumer (e.g. `audit`) is introduced as part of the same refactor, that consumer's module must declare `allowedDependencies = {..., "portfolios::events"}`.
- **What architectural assertion is added?** A bespoke test (`portfoliosEventsAreImmutableRecords`) that scans `portfolios.events` and asserts every type is a `record` is a strong candidate. Discuss whether to also assert no infrastructure imports.

5.3 Suggested package layout for the proposed solution

The exact layout is the participants' decision, but a defensible target – consistent with how Watchlists exposes its event – is:

```
application/src/main/java/cat/gencat/agaur/hexastock
    StockSoldEvent.java                    (or whicheve

bootstrap/src/main/java/cat/gencat/agaur/hexastock/p
    package-info.java                      @NamedInterf
```

```
application/src/main/java/cat/gencat/agaur/hexastock
    SaleTransactionRecordingListener.java   (the @Ap
    PortfolioStockOperationsService.java    (modifie
                                             no long
```

A natural alternative is to place the listener in a separate `portfolios.adapter.out.events` sub-package to underline that it is an *adapter-style* infrastructure concern. The choice is itself a discussion point.

5.4 Suggested event shape

A serviceable starting point – *which participants are encouraged to question and improve* – is the following record:

```
package
cat.gencat.agaur.hexastock.portfolios.events;

import cat.gencat.agaur.hexastock.marketdata.
model.market.Ticker;
import
cat.gencat.agaur.hexastock.model.money.Money;
import
cat.gencat.agaur.hexastock.model.money.Price;
```

```
import cat.gencat.agaur.hexastock.model.money.
ShareQuantity;
import cat.gencat.agaur.hexastock.portfolios.
model.portfolio.PortfolioId;

import java.time.Instant;
import java.util.Objects;

public record StockSoldEvent(
        PortfolioId portfolioId,
        String ownerName,
        Ticker ticker,
        ShareQuantity quantity,
        Price salePrice,
        Money proceeds,
        Money realisedProfit,
        Instant occurredOn
) {
    public StockSoldEvent {
        Objects.requireNonNull(portfolioId,
"portfolioId is required");
        Objects.requireNonNull(ownerName,
"ownerName is required");
        Objects.requireNonNull(ticker, "ticker is
required");
        Objects.requireNonNull(quantity, "quantity
is required");
        Objects.requireNonNull(salePrice,
"salePrice is required");
```

```
        Objects.requireNonNull(proceeds, "proceeds
is required");
        Objects.requireNonNull(realisedProfit,
"realisedProfit is required");
        Objects.requireNonNull(occurredOn,
"occurredOn is required");
    }
}
```

Whether `realisedProfit` should be modelled as a single signed `Money` or split into `realisedGain` / `realisedLoss` is a deliberate open question. Defer the answer to the participants.

5.5 Suggested publication site

Inside `PortfolioStockOperationsService.sellStock(...)`, after `portfolioPort.savePortfolio(portfolio)` and *before* the method returns:

```
SellResult sellResult = portfolio.sell(ticker,
quantity, price);
portfolioPort.savePortfolio(portfolio);

eventPublisher.publish(new StockSoldEvent(
        portfolioId,
        portfolio.getOwnerName(),
```

```
        ticker,
        quantity,
        price,
        sellResult.proceeds(),
        sellResult.profit(),
        clock.instant()));

return sellResult;
```

Two design notes worth discussing live:

- The publication is *after* `savePortfolio` and *inside* the `@Transactional` boundary. Spring Modulith's event publication registry binds the publication to the transaction; the registered listener fires only on commit.
- A `Clock` should be injected into the service rather than calling `Instant.now()` directly. The Watchlists module sets the precedent (MarketSentinelService accepts a `Clock` constructor argument).

5.6 Suggested consumer

```
package cat.gencat.agaur.hexastock.portfolios.
application.service;

import cat.gencat.agaur.hexastock.portfolios.
application.port.out.TransactionPort;
```

```java
import cat.gencat.agaur.hexastock.portfolios.events.StockSoldEvent;
import cat.gencat.agaur.hexastock.portfolios.model.transaction.Transaction;
import org.springframework.modulith.events.ApplicationModuleListener;
import org.springframework.stereotype.Component;

@Component
public class SaleTransactionRecordingListener {

    private final TransactionPort transactionPort;

    public SaleTransactionRecordingListener(TransactionPort transactionPort) {
        this.transactionPort = transactionPort;
    }

    @ApplicationModuleListener
    public void on(StockSoldEvent event) {
        Transaction tx = Transaction.createSale(
                event.portfolioId(),
                event.ticker(),
                event.quantity(),
                event.salePrice(),
                event.proceeds(),
                event.realisedProfit());
        transactionPort.save(tx);
    }
```

```
}
```

Two notes for the review session:

- `@Component` is acceptable here because the listener lives in the `application` Maven module's *test-and-runtime classpath* via Spring's component scanning. If ADR-007 should apply with the same strictness as elsewhere, place the listener in `bootstrap/.../config/events/` and wire it manually in `SpringAppConfig`. This is a legitimate trade-off worth discussing.
- The Transaction creation logic is now in two places at once during the migration window: it must be removed from `PortfolioStockOperationsService.sellStock(...)` *atomically with* the introduction of the listener, otherwise the integration tests will see two Transaction records per sale.

5.7 Suggested testing

Participants are expected to add at least:

1. **A unit test** for `SaleTransactionRecordingListener` that supplies a stub `TransactionPort`, invokes `on(...)` directly with a hand-built `StockSoldEvent` and asserts that one `Transaction` was

saved with the expected fields. No Spring context required.

2. **A modification to the existing service test** (PortfolioStockOperationsServiceTest – locate the file) so that the test now verifies that the `TransactionPort` is *not* called by the service (the responsibility has moved) and that the `DomainEventPublisher` *is* called with an event whose fields match the `SellResult`. Use a fake publisher that records published events.
3. **A new integration test** modelled exactly on NotificationsEventFlowIntegrationTest that boots the full Spring context, performs a sell through the application service and uses Awaitility to wait until exactly one `Transaction` is persisted by the listener. The test must assert:
   - The `Transaction` is recorded *after* the publishing transaction commits.
   - A failure inside the listener does *not* roll back the portfolio mutation.
4. **A new architecture assertion** that scans `cat.gencat.agaur.hexastock.portfolios.events` and asserts every type is a `record`.

---

6. Hexagonal-architecture boundaries to preserve

The refactor must respect the following invariants. Each of them is currently enforced by an existing test suite; the exercise will fail if any of them regresses.

Invariant	Enforced by
Domain types do not import Spring or JPA.	Maven module boundary + `HexagonalArchitectureTest`
Application services do not import Spring (other than annotations whitelisted in `application/pom.xml`).	Maven module boundary + ADR-007
`portfolios` does not import from `notifications`, `watchlists` or any other promoted module except `marketdata::model`, `marketdata::port-in`, `marketdata::port-out`.	`ModulithVerificationTest.port`
Inbound REST controllers do not call domain entities directly.	`HexagonalArchitectureTest$Ada`

Two points deserve emphasis with the participants:

- The new `StockSoldEvent` lives in the `application` Maven module *but inside the* ***`portfolios`*** *package*. It is therefore part of Portfolio Management's published API; the `@NamedInterface("events")` declaration on the bootstrap-side `package-info.java` is what makes that fact explicit to Spring Modulith.
- The new listener consumes a `portfolios` event and writes a `portfolios` aggregate (`Transaction`). This is *intra-module* and crosses no architectural boundary. Future consumers in other modules – `audit`, `reporting`, `integrations.brokerage` – would have to declare `allowedDependencies = {..., "portfolios::events"}` and would each be reviewed independently.

---

7. Stretch goal: the canonical *"one operation, many events"* pattern

The Sell Stocks use case under FIFO accounting consumes one or more `Lot` entities – possibly partially. Each consumption is a business fact in its own right:

- It has its own *cost basis* (the price at which the lot was originally purchased).
- It has its own *purchase date*, which matters for jurisdictions where holding-period rules affect tax treatment.
- It has its own *realised gain*, which is the per-lot contribution to the aggregate `SellResult.profit()`.

A natural extension of the exercise – for participants who finish the core refactor early or for a follow-up session – is to introduce a second event, `LotSoldEvent`, and to publish one `LotSoldEvent` per lot consumed alongside the single aggregate-level `StockSoldEvent`. The full design sketch lives in section 5.2 of the domain-events roadmap.

The mechanical change is in two places:

- The aggregate's `Portfolio.sell(...)` (or its delegate inside `Holding`) returns a richer `SellResult` that includes a `List<LotConsumption>` describing each per-lot consumption.
- The application service iterates that list and publishes one `LotSoldEvent` per element, immediately after the single `StockSoldEvent`.

This stretch goal is the most direct illustration of why event-driven design matters in a financial context: the aggregate operation is *one* (the user sold 100 shares); the business facts

emitted are *many* (one per lot consumed); each fact is independently consumable by reporting, audit and integration concerns. No alternative design known to the architecture community matches this expressiveness.

---

8. Out of scope (deliberate)

The following are explicitly *not* part of the exercise. Mention them so participants do not over-extend.

- Refactoring the buy path. The same pattern applies and the same exercise can be repeated, but doing both at once dilutes the focus.
- Externalising events to Kafka / RabbitMQ / JMS. Spring Modulith provides an in-process bus that is sufficient for HexaStock today; externalisation is a configuration change rather than a refactor and is out of scope here.
- Persisting the *event publication registry* with `spring-modulith-events-jpa` or `-mongodb`. The current configuration uses the in-memory registry, which is sufficient for the exercise and for development; production hardening is a separate task.

- Removing the `Transaction` aggregate. The `Transaction` aggregate remains. What changes is *who* writes it and *when*: not the application service synchronously, but a listener after commit.

---

9. Trade-offs the review must address

The facilitator should make sure the post-exercise review surfaces, at minimum, the following questions:

1. **Has the apparent simplicity of `sellStock(...)` been bought at the cost of *behavioural complexity* elsewhere?** The new asynchronous step adds operational concerns (event publication failures, listener exceptions, ordering, replays) that did not exist before. When is this trade favourable?
2. **Is the new event part of the public API of the `portfolios` module?** If yes, its shape becomes a contract; consumers will depend on its fields. Treat it as a published interface from day one.
3. **What happens if the listener throws?** With `@ApplicationModuleListener`, the listener's `REQUIRES_NEW` transaction rolls back; the publishing

transaction is unaffected. The event publication registry can be configured to retain the failed publication for re-delivery. Discuss whether the current in-memory registry is sufficient for production.

4. **Is the synchronous call to `transactionPort.save(...)` *really* a side effect, or is it part of the business invariant?** A defensible counter-argument is that *"every sale must be auditable"* is a business rule, and that breaking the synchronous link weakens it. The architectural answer is that the rule is preserved by the *event publication registry*: the publication is committed atomically with the portfolio mutation, and the audit record will be produced eventually. Whether *eventual* is acceptable for a given regulatory regime is a domain decision.
5. **When is *not* introducing a domain event the right answer?** If the only consumer is intra-aggregate, if the consumer must complete synchronously to satisfy a business invariant or if the cardinality of events would dwarf the benefit, the synchronous call is preferable. Domain events are not free; they pay back when reactions are plural, optional or eventually consistent.

A good review session ends with the participants able to articulate, in their own words, why HexaStock benefits from this refactor and what would have made it overengineering.

---

10. Instructor notes

10.1 Preparation

- Confirm that the participants have a working build: `./mvnw clean verify -DskipITs` should complete green. If anything fails on their machines, fix it before the session – the exercise depends on a green baseline.
- Open the eight files listed in Section 2 in IDE tabs and have them ready.
- Have a checked-out copy of NotificationsEventFlowIntegrationTest in front of you – participants will copy its structure for their own integration test.

10.2 Suggested timing (half-day session)

Slot	Activity
0:00-0:30	Instructor demo of the Market Sentinel flow (Section 2). Walk every artefact; insist on the five properties at the end of Section 2.
0:30-0:45	Joint reading of the current `sellStock(...)` (Section 3) and statement of the architectural problem (Section 4).
0:45-1:00	Participants split into pairs. Each pair drafts – *on paper* – the event name, the event payload and the package layout (Section 5.2 and 5.3). Compare answers across pairs before any code is written.
1:00-2:30	Implementation. The instructor circulates, answering questions but not coding. Participants follow Sections 5.4 to 5.7.

Slot	Activity
2:30-3:00	Each pair presents its solution. The instructor asks the trade-off questions of Section 9.
3:00-3:30	Optional stretch: introduce `LotSoldEvent` (Section 7) collectively at the screen.
3:30-4:00	Recap: when domain events help, when they hurt. Cross-reference to the domain-events roadmap for the rest of the platform.

10.3 What to look for during the review

- A `record` event with `Objects.requireNonNull` in the canonical constructor.
- No Spring import in the event class.
- The publisher injected as `DomainEventPublisher`, never `ApplicationEventPublisher`.
- The listener annotated with `@ApplicationModuleListener`, not with `@EventListener` or `@TransactionalEventListener` directly.

- A `Clock` injected into the service rather than `Instant.now()`.
- Existing tests still green.
- New integration test that uses Awaitility, mirrors `NotificationsEventFlowIntegrationTest` and asserts both the *positive* property (the listener fires) and the *negative* property (a listener failure does not corrupt the publishing transaction).

10.4 Common pitfalls

- **Removing the synchronous call before the listener works.** Insist on the listener being functionally complete *before* the synchronous call is removed; otherwise the integration tests turn red and the participants debug two problems at once.
- **Putting the event in the wrong package.** A common error is to place `StockSoldEvent` directly in `cat.gencat.agaur.hexastock.portfolios` (the module root). Doing so would expose it as the *default* published API. The exercise prefers a `events` sub-package with an explicit `@NamedInterface("events")` declaration, mirroring how `marketdata` exposes its three slices.
- **Confusing intra-module and cross-module listening.** This exercise is *intra-module*: the lis-

tener and the publisher both belong to `portfolios`. Clarify that this is legitimate and is a sound first step before introducing cross-module consumers in other modules.

- **Forgetting the time-source abstraction.** A test that asserts an exact `Instant` will be flaky if the service calls `Instant.now()`. Inject a `Clock`.

---

11. Where this fits in the wider documentation set

Document	Relationship to this exercise
doc/tutorial/sellStocks/SELL-STOCK-TUTORIAL.md	Walkthrough of the *current* Sell Stocks implementation. Read first.
doc/tutorial/sellStocks/SELL-STOCK-DOMAIN-TUTORIAL.md	Focused tutorial on the FIFO domain model. Background for Section 7.
doc/tutorial/sellStocks/SELL-STOCK-EXERCISES.md	Pre-existing self-directed exercises. This exercise is a natural addition to that catalogue.

Document	Relationship to this exercise
doc/tutorial/watchlists/WATCHLISTS-MARKET-SENTINEL.md	Reference for the instructor demo (Section 2).
doc/consultancy/04-DOMAIN-EVENTS-DEEP-DIVE.md	Complete dissection of the Market Sentinel event flow, beyond what the demo covers.
doc/consultancy/05-DOMAIN-EVENTS-ROADMAP.md	Forward-looking catalogue of further events; its section 5.2 (`LotSoldEvent`) is the source for the stretch goal of Section 7.
doc/architecture/MODULITH-BOUNDED-CONTEXT-INVENTORY.md	The promoted-module inventory. Useful when discussing future cross-module consumers.

---

12. Acceptance checklist for the participants' deliverable

A participant submission is *complete* when **all** of the following are true:

- ☐ A new event type (Java `record`, no framework imports) lives under `cat.gencat.agaur.hexastock.portfolios` and is published from `PortfolioStockOperationsService.sellStock(...)`.
- ☐ The synchronous `transactionPort.save(transaction)` call has been removed from `sellStock(...)`.
- ☐ A new `@ApplicationModuleListener` consumes the event and writes the `Transaction` record.
- ☐ A `Clock` is injected into `PortfolioStockOperationsService`.
- ☐ At least one new unit test covers the listener in isolation.
- ☐ The existing `PortfolioStockOperationsServiceTest` is updated to verify that the publisher is called and the transaction port is *not*.
- ☐ A new integration test mirrors `NotificationsEventFlowIntegrationTest` and verifies the after-commit, asynchronous Transaction recording with Awaitility.
- ☐ An ArchUnit assertion verifies that every type in the new events package is a `record`.
- ☐ `./mvnw clean verify -DskipITs` is green.
- ☐ `MODULES.verify()` and the bespoke Modulith assertions are green.

- ☐ A short Markdown note (one page) inside the participant's branch documents the design choices made in Section 5.2 and the trade-offs from Section 9.

A submission that meets all twelve items demonstrates that the participant has internalised the canonical domain-event pattern and is ready to apply it elsewhere in HexaStock without supervision.

05 – Instructor Guide for the Monday Session

> **Audience.** Yourself, on Monday morning, ten minutes before the room fills up. **Reading time.** ~10 minutes; refer back during the session.

---

1. Session objective

By the end of the session, every participant should be able to:

1. Explain how DDD, Hexagonal Architecture, and Spring Modulith coexist in HexaStock and what each style enforces.
2. Walk the Watchlists / Market Sentinel domain event flow end-to-end and defend the design choices in it.
3. Apply the same pattern themselves to the Sell Stocks use case during the hands-on exercise.
4. Articulate at least three realistic evolution paths beyond the current in-process design, and the trade-offs of each.

---

2. Suggested agenda (half-day, ~3.5 h with breaks)

Time	Block	Material
0:00 - 0:05	Welcome, framing, what we will *not* cover	Slide 1
0:05 - 0:30	HexaStock and the four architectural styles	00-ARCHITECTURE-OVERVIEW.md
0:30 - 1:00	Filesystem, Maven and Modulith structure (live tour)	01-FILESYSTEM-AND-MAVEN-STRUCTURE.md
1:00 - 1:15	**Break**	–
1:15 - 1:55	Watchlists / Market Sentinel deep dive (live demo)	02-WATCHLISTS-EVENT-FLOW-DEEP-DIVE.md

Time	Block	Material
1:55 - 2:20	Layout alternatives & trade-offs (whiteboard discussion)	03-LAYOUT-ALTERNATIVES.md
2:20 - 2:40	Production evolution path	04-PRODUCTION-EVOLUTION.md
2:40 - 2:50	**Break**	–
2:50 - 3:25	Hands-on exercise: Sell Stocks with Domain Events	SELL-STOCK-DOMAIN-EVENTS-EXERCISE.md
3:25 - 3:35	Group debrief on exercise	–
3:35 - 3:50	Wrap-up, Q&A, next steps	CHEATSHEET.md

If pressed for time, drop the production-evolution block to 10 minutes and keep everything else.

---

3. Recommended flow

3.1 Open with the *thesis*, not with the project

Spend the first five minutes stating what the project is *not*: not microservices, not event-sourced, not CQRS-everywhere, not asynchronous everything. The thesis: a *deliberately conservative* architectural shape that gives compile-time enforcement of two orthogonal things – *layer purity* and *bounded-context isolation* – without paying the operational cost of distribution.

3.2 Tour the filesystem live, with the IDE projector

Open the IDE on the projector and walk through:

```
domain/src/main/java/cat/gencat/agaur/hexastock/watc
application/src/main/java/cat/gencat/agaur/hexastock
application/src/main/java/cat/gencat/agaur/hexastock
bootstrap/src/main/java/cat/gencat/agaur/hexastock/w
info.java
adapters-outbound-
notification/src/main/java/cat/gencat/agaur/hexastoc
```

Each file is a different point on the same vertical slice. Make the audience *see* that the BC name is the second package segment in every Maven module.

3.3 Run the verification tests live

```
./mvnw -pl bootstrap test
-Dtest=ModulithVerificationTest
./mvnw -pl bootstrap test
-Dtest=HexagonalArchitectureTest
```

Then, for theatre value, **deliberately break the rule**: in `MarketSentinelService` add `import org.springframework.stereotype.Service;` and re-run `HexagonalArchitectureTest` so it fails publicly. Revert. The visceral demonstration of *the build refusing the architectural error* is the single most memorable moment of the session.

3.4 Run the integration test as the demo

`NotificationsEventFlowIntegrationTest` exercises the whole chain in a single JVM. Run it on the projector and walk through the log output as the event flows:

```
./mvnw -pl bootstrap test
-Dtest=NotificationsEventFlowIntegrationTest
```

Then read the test method top to bottom to convince the audience that what they just saw is indeed a real end-to-end exercise.

3.5 Do the layout-alternatives discussion as a discussion, not a lecture

Show the side-by-side diagram, then ask the room: *"which of A, B, C, D would you pick for a project of this size, and why?"*. Let the answers come. Steer toward the two main pivots: (i) what each layout enforces vs. what it asks you to trust, and (ii) at what team size each layout starts to pay off.

3.6 Set up the exercise *before* the break

Push participants to clone the repository (or pull the branch) during the break so the hands-on time is spent on the architecture, not on git.

---

4. Key talking points (memorise three of these, improvise the rest)

1. **"The build is the architecture's bouncer."** Maven module dependencies enforce hexagonal layering; `MODULES.verify()` enforces bounded-context isolation. Neither relies on review discipline.
2. **"`@ApplicationModuleListener` is just three annotations in a trench coat."** `@TransactionalEventListener(AFTER_COMMIT) + @Async + @Transactional(REQUIRES_NEW)`. Once you understand each piece, the transactional safety of the in-process event bus stops feeling magical.
3. **"The event carries business identity, not infrastructure identity."** The single most common rookie mistake is putting Telegram chat ids in the event. Don't. Resolve channels in the consumer.
4. **"The publisher does not know the consumer exists."** That is the property you are paying for, and the property the exercise tests.
5. **"In-process domain events are the *first* step of an evolution, not the *last* one."** Outbox, externalisation, extraction are all preserved options.
6. **"Bounded contexts are not microservices."** They are *modelling decisions*. Microservices are *de-*

ployment decisions. Conflating them is the textbook error this project avoids.

---

5. Demo sequence (cut-and-keep card)

1. Show the Maven dependency graph in the IDE.
2. Show two `package-info.java` files for the same `watchlists` package.
3. Show `MarketSentinelService` and walk the `detectBuySignals()` method.
4. Show `WatchlistAlertTriggeredEvent` (the record).
5. Show `WatchlistAlertNotificationListener` (the consumer).
6. Run `ModulithVerificationTest`.
7. Run `HexagonalArchitectureTest`.
8. Run `NotificationsEventFlowIntegrationTest`.
9. (Optional theatre) Break a rule, watch the build fail, revert.

---

6. Discussion prompts

Use these to draw the audience out:

- *Where in this codebase would you put a "user signed up" event, and why?*
- *Suppose Marketing wants every stock sale to trigger an email. Where do you change code? How does your answer differ between the current sell flow and the target sell flow?*
- *Should* `notifications` *know about* `marketdata`*? Today it does, because* `Ticker` *is in the event payload. Is that a leak?*
- *If you had to remove Spring Modulith tomorrow, what would you lose? What would you keep?*
- *What would you measure on Monday morning to know whether the in-process event bus is starting to cost you?*
- *At what team size does Layout C (per-BC mini-hexagons) start to be worth it?*

---

7. Common misconceptions to address proactively

Misconception	Correction
"Spring Modulith is a microservices framework."	It is a build-time module verifier and an in-process event bus with transactional semantics.
"Hexagonal means three concentric circles in a slide."	Hexagonal means *the application defines ports, the adapters implement them.* The visual is incidental.
"DDD requires aggregates everywhere."	DDD requires *thinking about* aggregates. Most code is not in an aggregate. The Watchlists aggregate is small for a reason.
"Domain events should be published from the aggregate."	They *can* be. Here they are published from the application service because the trigger condition (price-threshold) is *not* an aggregate invariant – it is a cross-aggregate read-side observation.

Misconception	Correction
"If we add `@Async`, we have made it scalable."	We have made it *off-thread*. Scalability requires bounded executors, back-pressure, and idempotent consumers.
"The outbox is what gives us exactly-once."	The outbox gives at-least-once with idempotent consumers. Exactly-once is a marketing term outside of constrained transactional contexts.

---

8. Architecture trade-offs to highlight (when asked)

- **Two `package-info.java` per Modulith package.** Yes, it is friction. It is the price of `application` staying Spring-free (ADR-007). The trade could be reversed if the team accepted a Spring import in the application layer.
- **Per-BC vertical slicing across many Maven modules.** Yes, the IDE experience is denser. The

pay-off is a build that refuses to compile layer violations.

- **In-process event bus.** Yes, it loses events on JVM crash. The evolution path to durable / brokered delivery is a configuration change, not a redesign – that is the point.

---

9. Suggested exercise flow (35 minutes)

Time	What
0:00 - 0:05	Read §3-§4 of the exercise document.
0:05 - 0:10	Discuss in pairs: *what is the business fact?*
0:10 - 0:25	Implement: define the event, publish it, write the listener.
0:25 - 0:30	Add the integration test. Make it green.
0:30 - 0:35	One pair presents their event payload to the room; group critiques.

Pre-warn participants that the exercise is *not* a code-completeness test – it is a discussion vehicle. The interesting outputs are *what they put in the payload, what they leave out, and where they put the listener.*

---

10. Wrap-up points (last five minutes)

1. The architecture is **conservative by design** – it favours enforcement over ergonomics.
2. The Watchlists flow is the **smallest correct example** of every concept covered.
3. The Sell Stocks exercise is the **same pattern, applied with one twist** (the consumer writes to the database).
4. Every "production-grade" property the audience asked about is a **preserved option**, not an architectural debt.
5. **Send the participants the doc map**: `doc/consultancy/monday-session/README.md`.

Optional homework: read `doc/consultancy/05-DOMAIN-EVENTS-ROADMAP.md` for a forward-looking catalogue of additional domain events the project will benefit from.

06 – Slide Deck Specification (AI-feedable)

Purpose. A structured, slide-by-slide specification suitable for direct ingestion by an AI slide-generation tool (Beautiful.ai, Gamma, Tome, Microsoft Designer, Google's NotebookLM slide export, etc.).

Audience of the resulting deck. Senior Java engineers and architects. Tone: precise, technical, free of marketing language.

Visual style. Clean, low-chrome, technical. White backgrounds. One diagram per architecture slide. Diagram references point to the rendered SVGs in `diagrams/Rendered/` and (for the exercise) in `../../tutorial/sellStocks/diagrams/Rendered/`. Use SVG when supported.

Suggested total length. 28 slides, ~45 minutes of speaking time.

How to feed this to a slide generator

Each slide block below contains the same fields:

- # – slide number.
- **Title** – slide title.
- **Objective** – the one thing the audience must take away.
- **Bullets** – concise content (3 - 5 bullets).
- **Visual** – what to put on the slide; a path to a rendered diagram or a prose description of a chart.
- **Speaker notes** – what to actually say.
- **Timing** – suggested speaking time.

You may pass the document verbatim, or extract the fields you need.

Slide 1

- **Title:** HexaStock – DDD, Hexagonal, Spring Modulith and Domain Events
- **Objective:** Frame the session and set expectations.
- **Bullets:**

  - A small, real, financial-portfolio backend used as a teaching asset
  - Combines four architectural styles deliberately
  - Half-day session; theory + live demo + hands-on exercise

- **Visual:** Title card. No diagram.
- **Speaker notes:** Welcome. State explicitly what we will *not* cover: microservices, event sourcing, CQRS-as-religion, message brokers in production. Emphasise: the architecture is *deliberately conservative*.
- **Timing:** 2 min.

Slide 2

- **Title:** What HexaStock is
- **Objective:** Anchor the business model in two sentences.
- **Bullets:**
  - Open a portfolio, deposit cash, buy and sell stocks (FIFO)
  - Maintain watchlists with price-threshold alerts -> notifications
  - Single Spring Boot deployable; JPA *or* MongoDB; Telegram or logging

- **Visual:** None (text card) or a thumbnail of the architecture overview.
- **Speaker notes:** Keep it brief. The point is to establish the *domain* vocabulary before introducing any architectural term.
- **Timing:** 2 min.

Slide 3

- **Title:** Four architectural styles, one codebase
- **Objective:** Introduce the thesis.
- **Bullets:**
  - DDD – *where* the boundaries lie
  - Hexagonal – *between which layers* the boundaries lie
  - Spring Modulith – *enforces* both at build time
  - Domain Events – let modules *react* without coupling
- **Visual:** `Rendered/01-architecture-overview.svg`
- **Speaker notes:** Four styles, four jobs. The combination is what matters; any one of them on its own would not do.
- **Timing:** 3 min.

Slide 4

- **Title:** The four bounded contexts
- **Objective:** Make the BCs concrete.
- **Bullets:**
  - `portfolios` – Portfolio aggregate, FIFO trading, Transaction ledger
  - `marketdata` – Ticker, StockPrice, MarketDataPort
  - `watchlists` – Watchlist aggregate, Market Sentinel scheduler
  - `notifications` – channels, destinations, senders
- **Visual:** `Rendered/05-bounded-context-map.svg`
- **Speaker notes:** Note the asymmetry: `notifications` depends on `watchlists` (because it consumes the event); the inverse never holds.
- **Timing:** 2 min.

Slide 5

- **Title:** Maven multi-module layout
- **Objective:** Show the physical shape of the codebase.

- **Bullets:**
  - 9 Maven modules: domain, application, adapters-*, bootstrap
  - `application` does *not* depend on Spring (ADR-007)
  - `bootstrap` is the only Spring Boot module
- **Visual:** `Rendered/02-maven-multimodule.svg`
- **Speaker notes:** The Maven graph is the layer-purity *enforcement mechanism*. A leak does not compile.
- **Timing:** 3 min.

Slide 6

- **Title:** Filesystem layout – three orthogonal axes
- **Objective:** Resolve the "why is the same package in many trees?" confusion.
- **Bullets:**
  - Hex layer -> Maven module
  - Bounded context -> top-level Java package
  - Adapter tech -> distinct adapter Maven module
- **Visual:** `Rendered/03-filesystem-layout.svg`
- **Speaker notes:** Internalise this slide and the rest of the session is a walk in the park.

- **Timing:** 3 min.

Slide 7

- **Title:** Spring Modulith application modules
- **Objective:** Show what Modulith adds on top.
- **Bullets:**
  - 4 modules detected automatically by package
  - `@ApplicationModule` declares allowed dependencies
  - `MODULES.verify()` fails the build on cycles or undeclared imports
- **Visual:** `Rendered/04-modulith-modules.svg`
- **Speaker notes:** This is the BC-isolation enforcement mechanism. Hexagonal enforces layers; Modulith enforces contexts.
- **Timing:** 3 min.

Slide 8

- **Title:** Hexagonal – per bounded context, not global
- **Objective:** Correct the most common architectural misreading.
- **Bullets:**

  - One hexagon per BC (today: 4 hexagons)
  - Primary ports: `...UseCase` interfaces
  - Secondary ports: `...Port` interfaces (repository, market data, event publisher)
  - Adapters live in their own Maven modules

- **Visual:** `Rendered/06-hexagonal-view.svg`
- **Speaker notes:** "Hexagonal" is a layering rule, not a single hexagon drawn for the whole app.
- **Timing:** 3 min.

Slide 9

- **Title:** Watchlists / Market Sentinel – what it does
- **Objective:** Set up the demo.
- **Bullets:**
  - Scheduler scans active watchlists
  - Reads current prices for distinct tickers
  - Emits `WatchlistAlertTriggeredEvent` per triggered alert
  - Notifications module dispatches per channel
- **Visual:** `Rendered/07-watchlist-sentinel-sequence.svg`
- **Speaker notes:** The flow is short; walk it verbally before showing the code.

- **Timing:** 3 min.

Slide 10

- **Title:** The event itself – a Java record
- **Objective:** Show what a *good* domain event looks like.
- **Bullets:**
  - `record WatchlistAlertTriggeredEvent(...)` – immutable, value-equal
  - Carries business identity (`userId`), not infrastructure identity
  - Validates non-null at construction
  - Timestamped at the source via injected `Clock`
  - Enum-typed `AlertType`, evolvable
- **Visual:** Code snippet of the record (~10 lines, monospace).
- **Speaker notes:** The five properties on the slide are the recurring pattern; if the audience remembers only one slide, this is it.
- **Timing:** 3 min.

Slide 11

- **Title:** Where the event lives
- **Objective:** Trace the publication path through the codebase.
- **Bullets:**
  - Event in `application/.../watchlists/`
  - Port: `DomainEventPublisher` (Spring-free)
  - Adapter: `SpringDomainEventPublisher` in `bootstrap/`
  - Consumer: `WatchlistAlertNotificationListener` in `notifications`
- **Visual:** `Rendered/08-watchlist-event-flow.svg`
- **Speaker notes:** The application layer never imports Spring. Only the adapter does.
- **Timing:** 2 min.

Slide 12

- **Title:** `@ApplicationModuleListener` = three annotations in a trench coat
- **Objective:** Demystify the listener annotation.
- **Bullets:**

  - `@TransactionalEventListener(AFTER_COMMIT)` – only fires on commit
  - `@Async` – off the publisher's thread
  - `@Transactional(REQUIRES_NEW)` – own transaction, own failure budget
  - This is what makes in-process events *safe*, not just *convenient*

- **Visual:** Three concentric annotation pills with the listener method inside; or just a code block.
- **Speaker notes:** This is the slide where senior engineers stop being sceptical. Take your time.
- **Timing:** 3 min.

Slide 13

- **Title:** Notification dispatch – pluggable senders
- **Objective:** Show how the consumer side handles channels.
- **Bullets:**
  - `NotificationRecipientResolver` aggregates destinations per user
  - `NotificationSender.supports(destination)` selects the sender
  - Telegram sender is profile-gated

  - Adding SMS = three new classes, zero changes upstream

- **Visual:** `Rendered/09-notification-flow.svg`
- **Speaker notes:** This is where the value of the asymmetric coupling becomes obvious.
- **Timing:** 3 min.

Slide 14

- **Title:** Live demo – the build refusing the rule
- **Objective:** Make the architectural enforcement *visceral.*
- **Bullets:**
  - Run `ModulithVerificationTest`
  - Run `HexagonalArchitectureTest`
  - Add an illegal Spring import -> watch the build fail
  - Revert
- **Visual:** A still of an IDE test runner showing red bars; or a terminal screenshot of the failing test.
- **Speaker notes:** Do not skip this. It is the most memorable five minutes of the session.
- **Timing:** 5 min.

Slide 15

- **Title:** Layout alternatives at a glance
- **Objective:** Set up the discussion.
- **Bullets:**
  - A. Layered, single Maven module
  - B. Package-by-feature, single Maven module
  - C. Per-BC mini-hexagons
  - D. Modulith-first (top-level package per module)
  - E. HexaStock today (Hex × BC, by Maven *and* by package)
- **Visual:** `Rendered/10-layout-alternatives.svg`
- **Speaker notes:** Show the slide, then *ask the room which they would pick.* Let the discussion happen.
- **Timing:** 4 min.

Slide 16

- **Title:** Why the current shape, in four lines
- **Objective:** Give the audience a quotable summary.
- **Bullets:**
  - Layer purity -> enforced by Maven modules

  - BC isolation -> enforced by Spring Modulith
  - Cost: one extra `package-info.java` per module + a denser Maven graph
  - Benefit: leaks fail to compile

- **Visual:** None or a quote card.
- **Speaker notes:** This is the *quotable* version of the thesis.
- **Timing:** 2 min.

Slide 17

- **Title:** What the in-process event bus actually guarantees
- **Objective:** Be honest about the limits.
- **Bullets:**
  - [OK] Delivered iff publisher's transaction commits
  - [OK] Listener does not slow down the publisher
  - [OK] Listener failure does not roll back the publisher
  - ☒ Survives a JVM crash
  - ☒ Has retries
- **Visual:** A two-column table ([OK] / [X]).

- **Speaker notes:** This is the slide that buys credibility with the ops/SRE-leaning members of the audience.
- **Timing:** 3 min.

Slide 18

- **Title:** Evolution path 1 – durability inside the monolith
- **Objective:** Show that the monolith can become *durable* without being rewritten.
- **Bullets:**
  - Add `spring-modulith-starter-jpa`
  - Events persisted to outbox in same transaction
  - Listener completion tracked, retried until success
  - Idempotency becomes a real concern
- **Visual:** Left half of `Rendered/11-current-vs-future-events.svg`
- **Speaker notes:** Single dependency change.
- **Timing:** 3 min.

Slide 19

- **Title:** Evolution path 2 – externalise to a broker
- **Objective:** Show the seam to async cross-process.
- **Bullets:**
  - `@Externalized("watchlist.alert.triggered.v1`
  - Event becomes a *published contract* – schema discipline begins
  - Out-of-process consumers become possible
- **Visual:** Middle of `Rendered/11-current-vs-future-events.svg`
- **Speaker notes:** The architecture *preserves* this option. It does not commit to it.
- **Timing:** 3 min.

Slide 20

- **Title:** Evolution path 3 – physical extraction
- **Objective:** Demystify the path to "microservices, when justified".
- **Bullets:**
  - Notifications can be its own deployable
  - Shared `Ticker` becomes a question (vendor / library / replace with String)

  - Operational concerns enter; modelling stays the same
  - This is a *capability you preserve*, not a *target you commit to*

- **Visual:** Right of `Rendered/11-current-vs-future-events.svg`
- **Speaker notes:** This is the slide where someone will say *"so why not just start with microservices?"* – be ready with: "because we'd be paying the cost without having validated that we need it".
- **Timing:** 3 min.

Slide 21

- **Title:** Hands-on exercise – Sell Stocks with Domain Events
- **Objective:** Set up the exercise.
- **Bullets:**
  - Today: `sellStock(...)` synchronously writes the `Transaction`
  - Goal: emit a `StockSoldEvent` and let a listener write the transaction
  - Reuse `DomainEventPublisher` and `@ApplicationModuleListener`

  - Same pattern as Watchlists, applied to a *write-side* consumer

- **Visual:** `sell-events-conceptual.svg`
- **Speaker notes:** Set the framing: this is the same pattern, with a twist (the consumer writes).
- **Timing:** 3 min.

Slide 22

- **Title:** Sell Stocks – current synchronous flow
- **Objective:** Show what we are refactoring.
- **Bullets:**
  - Controller -> service -> portfolio repo -> market data -> portfolio repo -> transaction repo
  - Two persistence side effects in one method
  - Adding any new reaction means modifying `sellStock(...)`
- **Visual:** `sell-events-current.svg`
- **Speaker notes:** Read the current method top to bottom.
- **Timing:** 3 min.

Slide 23

- **Title:** Sell Stocks – target event-driven flow
- **Objective:** Show the destination.
- **Bullets:**
  - `sellStock(...)` writes the portfolio and publishes the event
  - `SaleTransactionRecordingListener` writes the transaction (AFTER_COMMIT)
  - New reactions = new listeners, not edits to `sellStock(...)`
- **Visual:** `sell-events-target.svg`
- **Speaker notes:** Emphasise that the change is *additive*. Future reactions will not touch the method again.
- **Timing:** 3 min.

Slide 24

- **Title:** Exercise tasks (35 minutes)
- **Objective:** Hand the work to the room.
- **Bullets:**
  - Define `StockSoldEvent` (decide payload – *defend* it)

  - Replace the synchronous transaction write with a publication
  - Add `SaleTransactionRecordingListener`
  - Add an integration test mirroring `NotificationsEventFlowIntegrationTest`

- **Visual:** None or a checklist card.
- **Speaker notes:** Pair them up. Pre-warn: the exercise grade is *the decisions they defend*, not the line count.
- **Timing:** 1 min.

Slide 25

- **Title:** Common mistakes during the exercise
- **Objective:** Pre-empt the predictable.
- **Bullets:**
  - Putting persistence ids of the *Transaction* in the event payload
  - Calling the listener from inside the publishing transaction
  - Letting the listener throw (forgetting REQUIRES_NEW semantics)
  - Adding `@Component` on the application service
- **Visual:** None.

- **Speaker notes:** Walk the room. Catch these before they spread.
- **Timing:** 2 min.

Slide 26

- **Title:** Stretch – `LotSoldEvent` per FIFO consumption
- **Objective:** Tease the deeper pattern.
- **Bullets:**
  - One aggregate operation, multiple emitted events
  - Per-lot event enables tax-lot reporting downstream
  - Discussed in `doc/consultancy/05-DOMAIN-EVENTS-ROADMAP.md`
- **Visual:** None or a small fan-out diagram.
- **Speaker notes:** Optional content; mention only if time allows.
- **Timing:** 2 min.

Slide 27

- **Title:** What to take away

- **Objective:** Close the loop.
- **Bullets:**
  - The build enforces the architecture; reviews don't
  - Domain events are an *internal coordination mechanism* before they are an *integration mechanism*
  - In-process today, durable + brokered tomorrow, extractable the day after – same domain code
  - Bounded contexts are modelling decisions, not deployment decisions
- **Visual:** A short bullet card.
- **Speaker notes:** This is the slide they will photograph.
- **Timing:** 3 min.

Slide 28

- **Title:** Q&A and references
- **Objective:** Hand off to the room.
- **Bullets:**
  - `doc/consultancy/monday-session/` – this folder
  - `doc/consultancy/` – full briefing pack

  - `doc/architecture/MODULITH-BOUNDED-CONTEXT-INVENTORY.md`
  - `doc/architecture/adr/` – ADRs (especially ADR-007)
  - Spring Modulith reference

- **Visual:** A reference card with QR codes if printed.
- **Speaker notes:** Keep this slide up during Q&A.
- **Timing:** 5+ min Q&A.

Notes for the slide generator

- Prefer **SVG** over PNG when the target tool supports it (better scaling).
- Keep slide titles short. Bullet count: 3 - 5 lines, monospace for code.
- For diagram slides, the diagram is the slide; bullets go in speaker notes.
- For text slides, prefer a single quote-card layout.
- Footer on every slide: *HexaStock – DDD · Hexagonal · Spring Modulith · Domain Events.*